2.3–2.4 · Homogeneous transforms and forward kinematics; the Jacobian and singularities

Sessions 2.3–2.4 explain how joint positions determine a gripper's pose, how joint velocities determine its motion, and how to use that local motion map in reverse.

We build on the frames, rotations and change-of-frame equation from session 2.2:

$$ {}^{A}\mathbf{p}^{P} = {}^{A}\mathbf{p}^{B} + {}^{A}\mathbf{R}^{B}\,{}^{B}\mathbf{p}^{P}. $$

Here P is a point, and A and B are coordinate frames. The superscript on the left names the frame in which the coordinates are expressed. The new pose matrix will combine the rotation and translation in this equation.

The worked calculations use a two-link planar arm and an FR3 arm with a gripper. Optional sections provide further singularity analysis and Panda demonstrations.

0. What you will be able to do afterwards

- Write any pose as a 4×4 homogeneous transform, compose poses along a chain of frames by multiplying, and invert one without a matrix inversion.
- Read a robot as a kinematic tree, write its forward kinematics $\varphi(\mathbf{q})$ as a product of fixed link transforms and joint-dependent transforms, and evaluate it by hand for a two-link arm.
- Distinguish the current gripper pose computed from the joints from a desired grasp pose composed from a supplied object pose and a chosen object-relative grasp.
- Derive the Jacobian of that arm, check it by finite differences, explain what its columns and its determinant mean, and find where its position task loses a first-order motion direction.
- Solve the square differential inverse kinematics problem $\mathbf{J}\dot{\mathbf{q}} = \dot{\mathbf{x}}$, and say why the general problem (non-square, near-singular, with limits) needs the optimization of week 3.
- Explain why a velocity request can have several joint-velocity answers, what the pseudo-inverse selects, and how coordinated joint motion can preserve a chosen task.
- Pose the static-target update: from the remaining displacement $\mathbf{e} = \mathbf{y}^{*} - \varphi(\mathbf{q})$ and the local model $\mathbf{J}\Delta\mathbf{q} \approx \mathbf{e}$, take the square step, and say what Deck 04 derives about it (§4.12).

1. From joint measurements to gripper motion

To grasp a mug, the robot must locate its gripper relative to the mug. Joint encoders give joint positions. The robot model gives the fixed geometry between joints. Combining these inputs along the arm gives the gripper's current pose. This calculation is **forward kinematics** (FK).

Once the gripper reaches the mug, we may want to lift it straight upward. Each joint contributes a different hand motion, so the joints must move together. The **Jacobian** describes each joint's contribution at the current configuration. It lets us predict hand velocity from joint velocities, or solve for joint velocities that produce a requested hand motion.

The inverse calculation can have one answer, several answers, or no exact answer at that instant. We first solve a two-link example with one answer, then examine how the task and configuration change the possibilities. These local calculations prepare the optimization methods that choose joint changes toward a target in week 3.

2. Prerequisite ramp

Two tools are needed for the main teaching sequence. The optional material uses a third tool.

The derivative of a vector function. Suppose $\varphi : \mathbb{R}^n \to \mathbb{R}^m$ maps n joint angles to m task coordinates, such as the tip's position coordinates. Its derivative at $\mathbf{q}$ is the $m \times n$ matrix of partial derivatives $J_{ij} = \frac{\partial \varphi_i}{\partial q_j}$, and the first-order model of φ near $\mathbf{q}$ is $\varphi(\mathbf{q} + \Delta\mathbf{q}) \approx \varphi(\mathbf{q}) + \mathbf{J}(\mathbf{q})\Delta\mathbf{q}$. Column j of $\mathbf{J}$ is the direction and rate at which the output moves when only q_j changes. The only derivatives used today are $\frac{d \sin\theta}{d\theta} = \cos\theta$ and $\frac{d \cos\theta}{d\theta} = -\sin\theta$, together with the chain rule for $\sin(q_1 + q_2)$.

Rank, determinant, and the inverse of a 2 × 2 matrix. A square matrix is invertible exactly when its determinant is non-zero, and then

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}, \qquad ad - bc \neq 0.$$

The rank of a matrix is the number of linearly independent columns. A 2 × 2 matrix with parallel columns has rank 1 if at least one column is nonzero. If both columns are zero, its rank is 0. Both cases have determinant 0.

Optional ramp for singular values and the ellipse (§4.9; backup slide 55). The main sequence uses independent columns, rank, and the square inverse. It does not require this factorization. Any matrix with m rows and n columns has a singular value decomposition:

$$\mathbf{J} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^{\top}.$$

In the full decomposition, $\mathbf{U}$ is an orthogonal matrix with m rows and columns, and $\mathbf{V}$ is an orthogonal matrix with n rows and columns. Orthogonal matrices preserve lengths; they may rotate or reflect directions. The matrix $\boldsymbol{\Sigma}$ has m rows and n columns, with nonnegative entries on its main diagonal and zeros elsewhere. Those entries are the singular values, listed from largest to smallest. Two facts are used: $\mathbf{J}$ has rank r exactly when r singular values are non-zero, and for square $\mathbf{J}$, $|\det \mathbf{J}| = \sigma_1\sigma_2\cdots\sigma_n$. For the square two-link position Jacobian, the smallest singular value $\sigma_{\min}$ is the least tip speed over all unit joint-rate vectors. It is the weakest motion gain in the task's units.

A redundant arm can move its joints without moving the task at all. For a redundant Jacobian with full row rank, $\sigma_{\min}$ is the least gain over unit joint-rate vectors **orthogonal to the null space**. If we include null-space vectors, the least task speed is zero even though every listed singular value is positive. Section 4.11 explains that freedom. The smallest listed singular value is zero exactly when

$$\operatorname{rank}\mathbf{J} < \min(m, n).$$

This detects a singular configuration when that dimension bound is normally attainable. The general definition uses the maximum rank attainable by the chosen task (§3.4). A small singular value describes a weak motion gain, and its size depends on the units and scaling of the task. It does not measure joint-angle distance to a singular configuration. A determinant can be tiny because all columns are short, so inspect the individual singular values. NumPy computes them with `np.linalg.svd`.

The differential-IK and null-space explanations in §4.11 use the column interpretation and matrix rank. You can follow them without the optional singular-value calculations.

3. Formal setup

3.1 Homogeneous transforms

Append a 1 to every position vector: $\widetilde{\mathbf{p}} = (p_x, p_y, p_z, 1)^{\top}$. The **homogeneous transform** of frame B relative to frame A is the 4 × 4 matrix

$$ {}^{A}\mathbf{X}^{B} = \begin{bmatrix} {}^{A}\mathbf{R}^{B} & {}^{A}\mathbf{p}^{B} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}. \tag{1}$$

The set of all such matrices, with $\mathbf{R} \in \mathrm{SO}(3)$, is called $\mathrm{SE}(3)$ (the special Euclidean group). The pair $(\mathbf{R}, \mathbf{p})$ was already called ${}^{A}\mathbf{X}^{B}$ in 2.2; (1) is its matrix form.

3.2 Kinematic trees

A robot is a **tree of rigid links connected by joints** (a chain, for an arm). For the fixed-base arms of this block each joint is a hinge or a slider: it moves one body relative to its parent by a motion that depends on **one number**, the joint variable q_i: a **revolute** joint rotates about a fixed axis by the angle q_i; a **prismatic** joint slides along a fixed axis by the distance q_i. Spring 2026 deck 02 p.28 lists other joint types (spherical, planar, cylindrical). These allow combinations of rotations and translations, and a simulator may represent them with a multi-coordinate joint or a chain of simpler joints. A spherical or a free joint carries more than one coordinate, and a floating base has different dimensions for its coordinates and its velocities; none of that appears in our arms. The vector of all joint variables is the **configuration** $\mathbf{q} \in \mathbb{R}^n$, n the number of degrees of freedom (7 for the Franka Panda and Research 3, 2 for the planar arm below).

Between two consecutive joints there is a rigid link, whose transform is **constant**: it comes from the arm's geometry, not from $\mathbf{q}$. A RAI `.g` file is exactly this tree written down: every frame names its parent in parentheses, a constant offset is a `Q:` attribute, and a joint is a `joint:` attribute (`hingeX`, `hingeY`, `hingeZ` for revolute joints about the three axes, `transX`, `transY`, `transZ` for prismatic ones), with `q:` giving its initial value. This is verified against `rai-robotModels/scenarios/pendulum.g`, which reads `pend2joint (pend2base): { joint: hingeY, q:[1] }`. Our two-link arm is `code/two_link.g`.

3.3 Forward kinematics

Forward kinematics is the map from the configuration to the pose of a body of interest, usually the end-effector E:

$$\varphi : \mathbb{R}^n \longrightarrow \mathrm{SE}(3), \qquad \mathbf{q} \longmapsto {}^{W}\mathbf{X}^{E}(\mathbf{q}). \tag{2}$$

When only the position matters we write $\varphi(\mathbf{q}) = {}^{W}\mathbf{p}^{E}(\mathbf{q}) \in \mathbb{R}^3$ (or $\mathbb{R}^2$ for a planar arm), which is what the Goal box of the stack needs: it must find $\mathbf{q}$ with $\varphi(\mathbf{q}) = \mathbf{y}^{*}$.

3.4 The Jacobian, velocities, and singularities

For a position task, the output of forward kinematics is a coordinate vector. Its derivative is the **position Jacobian**:

$$\mathbf{J}(\mathbf{q}) = \frac{\partial \varphi}{\partial \mathbf{q}} \in \mathbb{R}^{m \times n}. \tag{3}$$

Here m is 3 in space or 2 in the plane. We also write this matrix as $\mathbf{J}_{\mathrm{pos}}$ to make the selected task explicit. Differentiating the position map in time gives

$$\mathbf{p}(t) = \varphi(\mathbf{q}(t)), \qquad \dot{\mathbf{p}} = \mathbf{J}_{\mathrm{pos}}(\mathbf{q})\dot{\mathbf{q}}.$$

For full rigid-body motion, we describe turning with the three-component angular velocity and motion of the selected frame's origin with the three-component linear velocity. We stack these as a spatial velocity, angular first, and stack their two Jacobian blocks:

$$ {}^{W}\mathbf{V}^{E} = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix} = \underbrace{\begin{bmatrix} \mathbf{J}_{\text{ang}} \\ \mathbf{J}_{\text{pos}} \end{bmatrix}}_{\mathbf{J} \in \mathbb{R}^{6 \times n}} \dot{\mathbf{q}}. $$

Both velocity vectors are expressed in world axes, and the linear vector refers to the origin of frame E. This geometric Jacobian maps joint velocities to those six motion components. It is not the derivative of six Euler-angle and position coordinates.

Reaching the right point is not enough for a grasp. The fingers must also face the intended grasp direction. Angular velocity describes how we turn them toward that orientation.

Angular velocity describes how the end-effector turns at the current instant. Its direction points along the instantaneous rotation axis, with the rotation sense given by the right-hand rule. It does not locate that axis line in space. The vector's magnitude is the angular speed:

$$ \text{angular speed} = \|\boldsymbol{\omega}\|. $$

Both vectors are expressed in world axes in this course. For the planar arm, the gripper turns only about the world z-axis, so its heading rate is the only component that can be nonzero:

$$ \boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{\theta} \end{bmatrix}. $$

Either sign is possible for the heading rate. A zero angular-velocity vector means no instantaneous turning and gives no unique axis direction. Angular velocity is generally not a vector of Euler-angle derivatives. For our fixed-base hinge-joint arms, the simulator's generalized joint velocities are the joint angular velocities. Floating-base coordinate-to-velocity conversions are deferred.

Constructing the angular block from the joint axes. At the current joint angles, express each hinge's positive rotation-axis direction in world coordinates. The existing forward-kinematics chain supplies the required rotation matrices. This converts the coordinates of a direction; it does not compute a point on the axis line.

In our FR3 model, each hinge's local unit axis is z. Its child-frame rotation therefore gives its world-axis direction by

$$ {}^{W}\hat{\mathbf{z}}_i = {}^{W}\mathbf{R}^{i}(\mathbf{q}) \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. $$

This is the third column of that rotation matrix. The hat marks a unit vector. The subscript identifies the joint; it does not mean the joint points along world z. For a model with a different declared local axis, rotate that unit vector instead.

Freeze all other joints and turn hinge i at one radian per second. The end-effector turns about that same axis at one radian per second. Thus its angular-Jacobian column is the axis direction. For a seven-hinge arm,

$$ \mathbf{J}_{\text{ang}}(:, i) = {}^{W}\hat{\mathbf{z}}_i, \qquad \boldsymbol{\omega} = \sum_{i=1}^{7} {}^{W}\hat{\mathbf{z}}_i \, \dot{q}_i. $$

All contributions use world coordinates before we add them. A sliding joint changes no orientation, so its angular column is zero. Where the hinge axis passes in space affects linear velocity, but not its angular contribution.

A numerical FR3 example. Section 5(m.1) evaluates the full transform chain at this ready configuration. Here we use its first two rotation blocks to construct angular columns:

$$ \mathbf{q} = (0, -\pi/4, 0, -3\pi/4, 0, \pi/2, \pi/4)^{\top}. $$

The second-link world rotation computed in that example is

$$ {}^{W}\mathbf{R}^{2} = \begin{bmatrix} c & c & 0 \\ 0 & 0 & 1 \\ c & -c & 0 \end{bmatrix}, \qquad c = \frac{\sqrt{2}}{2}. $$

Rotate the model's local axis into world coordinates:

$$ {}^{W}\hat{\mathbf{z}}_2 = \begin{bmatrix} c & c & 0 \\ 0 & 0 & 1 \\ c & -c & 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}. $$

Joint 1's fixed placement translates by 0.333 m without rotating. Its joint angle is zero at the ready pose, so frame 1 has the identity rotation relative to the world:

$$ {}^{W}\mathbf{R}^{1} = \mathbf{I}_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}. $$

Apply the same axis conversion to joint 1:

$$
{}^{W}\hat{\mathbf{z}}_1 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.
$$

Thus its positive axis is world z. We have now computed both directions, so the first two angular columns are

$$
\begin{bmatrix} {}^{W}\hat{\mathbf{z}}_1 & {}^{W}\hat{\mathbf{z}}_2 \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}.
$$

Move these joints at 0.1 and 0.2 rad/s while holding the other five still:

$$
\dot{\mathbf{q}} = (0.1, 0.2, 0, 0, 0, 0, 0)^{\top}\ \mathrm{rad/s}.
$$

Multiply the selected columns by those angular velocities:

$$
\boldsymbol{\omega} = \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.2 \\ 0.1 \end{bmatrix}\ \mathrm{rad/s}.
$$

The other five Jacobian columns still exist, but contribute nothing because their joint velocities are zero. This result is one instantaneous angular-velocity vector. It is not a sequence of finite rotations. For the planar arm both axes point along world z, so the same construction gives

$$
\mathbf{J}_{\text{ang}} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 1 \end{bmatrix}, \qquad \boldsymbol{\omega} = \begin{bmatrix} 0 \\ 0 \\ \dot{q}_1 + \dot{q}_2 \end{bmatrix}.
$$

Run `drake/01c-fr3-grasp-chain/fr3_grasp_chain.py` to reproduce the FR3 angular columns and check them against Drake at the ready pose and a changed configuration.

A configuration $\mathbf{q}$ is **singular for a chosen task** when its Jacobian has lower rank than the maximum rank that same task Jacobian attains anywhere on the robot (essentials §8). When that maximum is $\min(m, n)$, as for the planar position task, the test is $\operatorname{rank} \mathbf{J} < \min(m, n)$; for square $\mathbf{J}$ it is $\det \mathbf{J} = 0$. The optional singular-value test is $\sigma_{\min}(\mathbf{J}) = 0$ whenever the Jacobian normally has full row or column rank. At a singular configuration there is a direction of tip motion that no joint velocity produces at first order. The 3×2 position-and-heading Jacobian of §4.8 never has rank 3; that is task deficiency, not a singularity, so always say which task Jacobian is meant.

Four different questions. Positions tell us where the robot is. Velocities tell us how it is moving now. Keep the given quantities and the unknowns separate:

Question	Given	Compute or seek
Forward kinematics	Joint positions	Tip position or pose
Inverse kinematics	Desired tip position or pose	Joint positions that achieve it
Differential forward kinematics	Current joint positions and joint velocities	Tip linear velocity, or full spatial velocity
Differential inverse kinematics	Current joint positions and requested tip velocity	Joint velocities that produce that motion

The current joint positions determine the Jacobian in both velocity questions. For the position task, its output is the tip's linear velocity. For the full-pose task, the output has three angular and three linear components; the linear part is the velocity of the selected frame's origin. Neither velocity calculation differentiates sixteen independent pose-matrix entries. The inputs for our hinge-joint arms are joint angular velocities. A sliding joint would instead contribute a linear velocity. An inverse request can be unreachable or nonunique, so seeking an answer does not imply one always exists.

4. Derivation

4.1 Why the extra 1: the change-of-frame equation becomes a matrix product

The change-of-frame equation of 2.2, ${}^{A}\mathbf{p}^{P} = {}^{A}\mathbf{p}^{B} + {}^{A}\mathbf{R}^{B}\,{}^{B}\mathbf{p}^{P}$, is not a matrix–vector product: it has a matrix part and an added vector. With the extra coordinate,

$$\begin{aligned}\begin{bmatrix} {}^{A}\mathbf{R}^{B} & {}^{A}\mathbf{p}^{B} \\ \mathbf{0}^{\top} & 1 \end{bmatrix} \begin{bmatrix} {}^{B}\mathbf{p}^{P} \\ 1 \end{bmatrix} &= \begin{bmatrix} {}^{A}\mathbf{R}^{B}\,{}^{B}\mathbf{p}^{P} + {}^{A}\mathbf{p}^{B} \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} {}^{A}\mathbf{p}^{P} \\ 1 \end{bmatrix}. \end{aligned} \tag{4}$$

The first three rows give the change-of-frame equation, and the fourth gives $1 = 1$. Thus ${}^{A}\widetilde{\mathbf{p}}^{P} = {}^{A}\mathbf{X}^{B}\,{}^{B}\widetilde{\mathbf{p}}^{P}$: one product, and the reason for the odd-looking fourth row.

4.2 Composition

Apply (4) twice. A point with coordinates ${}^{C}\mathbf{p}^{P}$ in C has coordinates ${}^{B}\mathbf{X}^{C}\,{}^{C}\widetilde{\mathbf{p}}^{P}$ in B, hence ${}^{A}\mathbf{X}^{B}\left({}^{B}\mathbf{X}^{C}\,{}^{C}\widetilde{\mathbf{p}}^{P}\right)$ in A. Matrix multiplication is associative, so

$${}^{A}\mathbf{X}^{C} = {}^{A}\mathbf{X}^{B}\,{}^{B}\mathbf{X}^{C}. \tag{5}$$

with the inner letters matching, as for rotations in 2.2. Multiplying the blocks out confirms the two halves: rotation ${}^{A}\mathbf{R}^{C} = {}^{A}\mathbf{R}^{B}\,{}^{B}\mathbf{R}^{C}$ and position ${}^{A}\mathbf{p}^{C} = {}^{A}\mathbf{p}^{B} + {}^{A}\mathbf{R}^{B}\,{}^{B}\mathbf{p}^{C}$, which is the change-of-frame equation applied to the origin of C.

4.3 Inverse

${}^{B}\mathbf{X}^{A}$ must undo ${}^{A}\mathbf{X}^{B}$. Try the candidate

$$\begin{aligned} ({}^{A}\mathbf{X}^{B})^{-1} &= \begin{bmatrix} \mathbf{R}^{\top} & -\mathbf{R}^{\top}\mathbf{p} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}, \\ \mathbf{R} = {}^{A}\mathbf{R}^{B}, &\qquad \mathbf{p} = {}^{A}\mathbf{p}^{B}. \end{aligned} \tag{6}$$

Check by block multiplication: the rotation block of the product is $\mathbf{R}\mathbf{R}^{\top} = \mathbf{I}$, the position block is $\mathbf{R}(-\mathbf{R}^{\top}\mathbf{p}) + \mathbf{p} = -\mathbf{p} + \mathbf{p} = 0$, and the last row is $(0^{\top}, 1)$. So (6) is the inverse: transpose the rotation, then rotate and negate the translation. Transposing the whole pose matrix would be wrong; a generic matrix inverse is unnecessary for a valid rigid transform. For example, ${}^{E}\mathbf{X}^{W}$ expresses the world frame relative to the end-effector. Its position block gives the world origin's coordinates in the end-effector's axes.

4.4 Forward kinematics as a product of transforms

Walk from the world frame to the end-effector along the chain. Each step is either a constant link transform or a joint transform that depends on one joint coordinate. Let J_i name the frame just before joint i moves, and L_i the frame just after its motion. These are frame names; J_i here is not a Jacobian column. The transform from J_i to L_i depends on q_i. The transform from L_i to the next joint frame is fixed:

$$\begin{aligned} {}^{W}\mathbf{X}^{E}(\mathbf{q}) = {}^{W}\mathbf{X}^{J_1}\,{}^{J_1}\mathbf{X}^{L_1}(q_1)\,{}^{L_1}\mathbf{X}^{J_2} \\ {}^{J_2}\mathbf{X}^{L_2}(q_2)\cdots{}^{J_n}\mathbf{X}^{L_n}(q_n)\,{}^{L_n}\mathbf{X}^{E}. \end{aligned} \tag{7}$$

For a revolute joint about its own z axis and a prismatic joint along its own x axis,

$$\begin{aligned} \mathbf{X}_{\text{rev}}(q_i) &= \begin{bmatrix} \mathbf{R}_z(q_i) & \mathbf{0} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}, \\ \mathbf{X}_{\text{pri}}(q_i) &= \begin{bmatrix} \mathbf{I}_3 & (q_i, 0, 0)^{\top} \\ \mathbf{0}^{\top} & 1 \end{bmatrix}. \end{aligned} \tag{8}$$

Robot libraries evaluate this chain from the model. Drake's `plant.CalcRelativeTransform` returns the relative pose; RAI's `C.getFrame('endeff').getPosition()` returns the end-effector's world position.

The two-link planar arm. Joint 1 at the world origin, axis z, angle q_1; link 1 of length l_1 along the rotated x axis; joint 2 at its end, axis z, angle q_2; link 2 of length l_2; the end-effector frame E at its tip. Everything happens in the x–y plane, so we use the planar 3×3 versions of (1) and (8),

$$\begin{aligned}\mathbf{R}(\theta) &= \begin{bmatrix}\cos\theta & -\sin\theta & 0\\ \sin\theta & \cos\theta & 0\\ 0 & 0 & 1\end{bmatrix},\\ \mathbf{T}(x,y) &= \begin{bmatrix}1 & 0 & x\\ 0 & 1 & y\\ 0 & 0 & 1\end{bmatrix}.\end{aligned} \tag{9}$$

and (7) becomes

$$^{W}\mathbf{X}^{E}(\mathbf{q}) = \mathbf{R}(q_1)\,\mathbf{T}(l_1,0)\,\mathbf{R}(q_2)\,\mathbf{T}(l_2,0). \tag{10}$$

Evaluate the product from left to right by appending each factor on the right. Every joint angle is already fixed while we calculate. These are steps in evaluating one configuration, not commands to move the robot through four configurations.

Frame 1 is at the base with its x-axis along link 1. Frame 2 is at the elbow with its x-axis along link 2. Frame E is at the tip with the same heading as frame 2. The physical frame grouping is

$$^{W}\mathbf{X}^{E} = {}^{W}\mathbf{X}^{1}\,{}^{1}\mathbf{X}^{2}\,{}^{2}\mathbf{X}^{E} = \mathbf{R}(q_1)\,[\mathbf{T}(l_1,0)\mathbf{R}(q_2)]\,\mathbf{T}(l_2,0).$$

The four factors separate a joint rotation from a link offset so that we can inspect each calculation. They do not require four different physical links. After translating along link 1 but before appending the second rotation, we have a helper frame at the elbow whose axes still follow link 1. It is not the named frame 2 yet.

To see why the offset turns with the chain, multiply one pair of blocks. Here $\mathbf{R}_2$ is the 2×2 rotation block, while $\mathbf{R}(\theta)$ in (9) is a full 3×3 planar homogeneous transform:

$$\begin{bmatrix}\mathbf{R}_{2,\text{old}} & \mathbf{p}_{\text{old}}\\ \mathbf{0}^{\top} & 1\end{bmatrix}\begin{bmatrix}\mathbf{R}_{2,\text{rel}} & \mathbf{p}_{\text{rel}}\\ \mathbf{0}^{\top} & 1\end{bmatrix} = \begin{bmatrix}\mathbf{R}_{2,\text{old}}\mathbf{R}_{2,\text{rel}} & \mathbf{p}_{\text{old}} + \mathbf{R}_{2,\text{old}}\mathbf{p}_{\text{rel}}\\ \mathbf{0}^{\top} & 1\end{bmatrix}.$$

The relative offset is written in the current frame's axes. Its rotation block converts that offset into world axes, then the current origin is added. A pure rotation has zero relative offset, so it leaves this origin unchanged. A pure translation has identity relative rotation, so it leaves the heading unchanged. The numerical stages in §5(a) evaluate both rules one factor at a time.

Thus the first translation locates the elbow along link 1. The second rotation gives link 2 its world heading, and the final translation follows that link to the tip. The **closed form** is

$$\varphi(\mathbf{q}) = {}^{W}\mathbf{p}^{E} = \begin{bmatrix}l_1\cos q_1 + l_2\cos(q_1+q_2)\\ l_1\sin q_1 + l_2\sin(q_1+q_2)\end{bmatrix}, \qquad \theta = q_1 + q_2. \tag{11}$$

The heading is the direction of the end-effector's x axis: rotations about a common axis add.

4.4.1 Compute the current hand pose and construct a desired grasp separately

An arm model often ends at the flange where a hand is mounted. In the FR3 example, frame E is that flange and frame G is the hand's tool-center point between the fingers. The arm's joint-dependent chain reaches the flange. A fixed mounting transform reaches the tool-center point:

$$^{W}\mathbf{X}^{G}(\mathbf{q}) = {}^{W}\mathbf{X}^{E}(\mathbf{q})\,{}^{E}\mathbf{X}^{G}.$$

Both the mounting rotation and the offset matter. Omitting the offset would compare a flange point with a target intended for the fingers. Omitting the mounting rotation would compare different gripper axes even if the two origins happened to agree.

To place a gripper relative to a mug, we also need a target. Let M be the mug frame, with its origin at the mug body's center. This first scene supplies its world pose. A selected grasp specifies a desired gripper frame G^* relative to the mug:

$$^{W}\mathbf{X}^{G^*} = {}^{W}\mathbf{X}^{M}\,{}^{M}\mathbf{X}^{G^*}.$$

The mug pose answers where the object is. The object-relative grasp specifies where and how the gripper should meet it. The two pieces are separate inputs. A known object pose alone does not choose a grasp. The target frame is a desired copy of the same tool frame used for the current pose.

Before closing the hand, choose another target behind the grasp along its approach axis. For a stand-off distance d, the relative transform is

$$^{G^*}\mathbf{X}^{G^*_{\text{app}}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -d \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad {}^{W}\mathbf{X}^{G^*_{\text{app}}} = {}^{W}\mathbf{X}^{G^*}\,{}^{G^*}\mathbf{X}^{G^*_{\text{app}}}.$$

Here the offset is measured along the desired gripper's local z-axis. Its world direction comes from the desired rotation, just as the planar link offset did. Section 5(m) shows a downward-pointing grasp axis, so a negative local-z offset puts the approach target above the mug.

Forward kinematics evaluates the current pose from known joints. The two target products construct desired poses from scene and grasp inputs. Finding joint angles that make these poses agree is the inverse-kinematics question:

$$^{W}\mathbf{X}^{G}(\mathbf{q}^*) = {}^{W}\mathbf{X}^{G^*}.$$

If a later solver instead uses the flange frame as its output, convert the desired tool pose into a desired flange pose first:

$$ {}^{W}\mathbf{X}^{E^*} = {}^{W}\mathbf{X}^{G^*}\left({}^{E}\mathbf{X}^{G}\right)^{-1}. $$

The same fixed mounting geometry is used at the current and desired poses. This conversion changes which robot frame the target describes; it does not solve for the joints.

Writing this equation does not solve it or establish that its target is reachable. The new example stops at the two chains and their difference. The existing position-only examples and later optimization sessions supply the solver machinery. Collision-free motion and successful contact require further checks beyond a target pose.

How the course solves IK. Inverse kinematics names the target-to-joints problem. The target may have no answer, one answer, or several answers. For the ideal planar arm with link lengths 0.4 m and 0.3 m, its tip can reach distances between 0.1 m and 0.7 m from the base. A position outside this annulus has no solution. A position on either boundary has one geometric solution, and an interior position has two: one with a positive elbow angle and one with a negative elbow angle. These counts ignore joint bounds and obstacles, and identify angles that differ by full turns.

For the position target used by the next deck, the two joint configurations are

$$ \mathbf{y}^* = \begin{bmatrix} 0.5 \\ 0.2 \end{bmatrix}\ \mathrm{m}, \qquad \mathbf{q}_A \approx \begin{bmatrix} -11.52^\circ \\ 80.41^\circ \end{bmatrix}, \qquad \mathbf{q}_B \approx \begin{bmatrix} 55.12^\circ \\ -80.41^\circ \end{bmatrix}. $$

Either configuration places the tip at the target, but their headings differ. We have not prescribed heading in this position-only problem. These are two separate answers, not the continuous posture freedom of a redundant arm.

Analytic IK derives geometry-specific formulas; the two-link law-of-cosines example is in long reading §7.1. An analytic result can supply a candidate or a useful starting guess. It does not by itself establish that the motion to that configuration avoids collisions.

We develop numerical IK because a common optimization formulation can use each robot's FK map and accommodate posture preferences, joint limits and clearance constraints. This is why the next mathematical tool is the Jacobian. It predicts the effect of a small joint change; Deck04 derives how to choose an error-reducing change and Deck05 adds constraints. Differential IK solves a related velocity question at the current configuration. Repeating local displacement updates can improve a static IK guess, but the methods are local and need not converge from every start. The iterative solver remains a backup preview here; its derivation stays in the optimization block.

4.5 The Jacobian of the two-link arm

Differentiate (11) with $\frac{d\sin\theta}{d\theta} = \cos\theta$, $\frac{d\cos\theta}{d\theta} = -\sin\theta$, and $\frac{\partial(q_1+q_2)}{\partial q_1} = \frac{\partial(q_1+q_2)}{\partial q_2} = 1$:

$$\begin{aligned}\mathbf{J}(\mathbf{q}) &= \begin{bmatrix} \dfrac{\partial x}{\partial q_1} & \dfrac{\partial x}{\partial q_2} \\ \dfrac{\partial y}{\partial q_1} & \dfrac{\partial y}{\partial q_2} \end{bmatrix} \\ &= \begin{bmatrix} -l_1 \sin q_1 - l_2 \sin(q_1 + q_2) & -l_2 \sin(q_1 + q_2) \\ l_1 \cos q_1 + l_2 \cos(q_1 + q_2) & l_2 \cos(q_1 + q_2) \end{bmatrix}.\end{aligned} \tag{12}$$

With the current joint positions known, multiply the Jacobian by the joint velocities to predict the resulting tip velocity:

$$\dot{\mathbf{p}} = \mathbf{J}_{\text{pos}}(\mathbf{q})\dot{\mathbf{q}}.$$

This is **differential forward kinematics**. The inputs are the current joint positions and the joint angular velocities; the output is tip velocity. The calculation is a matrix product. Differential IK later reverses the velocity question and solves for the joint angular velocities. Section5(d) evaluates the forward calculation numerically.

4.6 What the columns mean

Hold joint 2 fixed and turn joint 1. The entire arm rotates rigidly about the shoulder, so the tip follows a circle centered at the shoulder. Its instantaneous motion is tangent to that circle, perpendicular to the shoulder-to-tip radius. In this example the shoulder is at the world origin, so that radius has coordinates (x, y). Rotating the radius vector counterclockwise by 90° gives the first column:

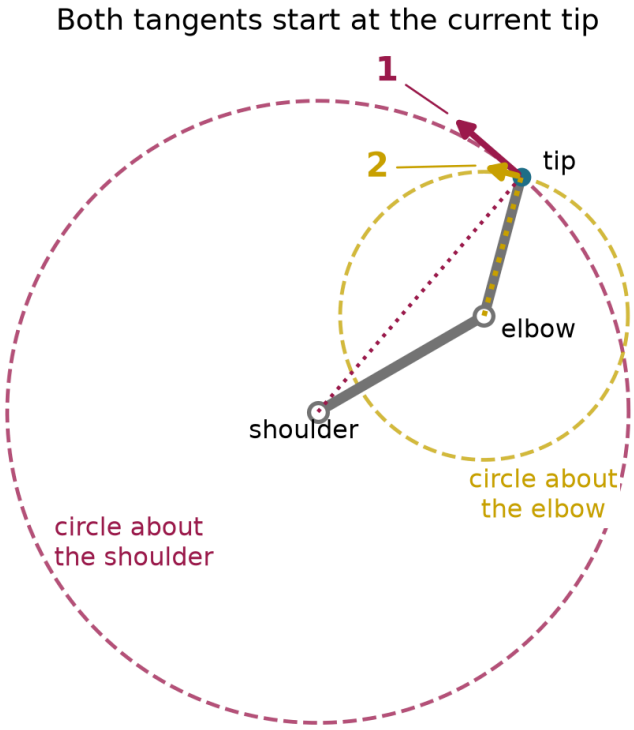
$$\frac{\partial \mathbf{p}}{\partial q_1} = \begin{bmatrix} -y \\ x \end{bmatrix}.$$

This rotation describes how to construct the tangent vector. It does not mean that the tip actually moves through 90°. The tangent is perpendicular to the shoulder-to-tip line, which generally differs from the direction of link 1.

Now hold joint 1 fixed and turn joint 2. The elbow stays still, and the tip follows a circle centered at the elbow. This circle's radius is link 2. The second column is perpendicular to that link:

$$\frac{\partial \mathbf{p}}{\partial q_2} = l_2 \begin{bmatrix} -\sin(q_1 + q_2) \\ \cos(q_1 + q_2) \end{bmatrix}.$$

Each column gives the tip's direction and first-order motion per radian of the selected joint. Its magnitude equals the corresponding circle's radius. A larger radius gives more tip motion for the same small turn. Multiplying the column by the joint's small angle change gives a predicted displacement; multiplying it by the joint's angular rate gives a velocity.



The maroon circle is the tip's path when only joint 1 turns. The gold circle is its path when only joint 2 turns. Both circles pass through the current tip. Arrows 1 and 2 show their positive tangents; they are the Jacobian columns scaled by 0.3 radians for display, not exact finite displacements.

Every column of a position kinematic Jacobian has this form for a revolute joint: **axis direction crossed with the vector from the joint to the tip**, $\frac{\partial \mathbf{p}}{\partial q_i} = \hat{\mathbf{z}}_i \times (\mathbf{p} - \mathbf{p}_{\text{joint } i})$, which is how Drake and RAI compute Jacobians for a 7-joint arm without symbolic differentiation.

4.7 The determinant, and where it vanishes

From (12), with the abbreviations $s_1 = \sin q_1$, $c_1 = \cos q_1$, $s_{12} = \sin(q_1 + q_2)$, $c_{12} = \cos(q_1 + q_2)$:

$$\begin{aligned} \det \mathbf{J} &= (-l_1 s_1 - l_2 s_{12})(l_2 c_{12}) - (-l_2 s_{12})(l_1 c_1 + l_2 c_{12}) \\ &= -l_1 l_2 s_1 c_{12} - l_2^2 s_{12} c_{12} + l_1 l_2 s_{12} c_1 + l_2^2 s_{12} c_{12} \\ &= l_1 l_2 (s_{12} c_1 - c_{12} s_1) \\ &= l_1 l_2 \sin((q_1 + q_2) - q_1) = l_1 l_2 \sin q_2 . \end{aligned} \tag{13}$$

using the angle-difference identity $\sin(a - b) = \sin a \cos b - \cos a \sin b$ in the last step. The l_2^2 terms cancel. So $\det \mathbf{J} = 0$ exactly when $\sin q_2 = 0$: the arm **stretched** ($q_2 = 0$) or **folded** ($q_2 = \pm 180^\circ$). At those configurations both columns of (12) are parallel, both perpendicular to the line from the base to the tip, and no joint velocity moves the tip **along** that line at first order. The tip then sits on the boundary of the reachable workspace, the annulus between radii $|l_1 - l_2|$ and $l_1 + l_2$, which is the

geometric reason: you cannot move outward from the outer boundary or inward from the inner one. q_1 does not appear in (13): the base joint changes where the tip is, not how well it moves.

4.8 The angular part, and counting

Position alone does not say which way a gripper attached to link 2 points. Add its heading to the two position coordinates to describe its planar pose:

$$\begin{bmatrix} \theta \\ x \\ y \end{bmatrix}, \qquad \theta = q_1 + q_2.$$

These are one angle and two coordinates in the plane, not a position in 3D. First differentiate the heading with respect to each joint, holding the other fixed:

$$\frac{\partial \theta}{\partial q_1} = 1, \qquad \frac{\partial \theta}{\partial q_2} = 1.$$

The dotted joint coordinates are the joint velocities. Because both joints are revolute, these are specifically **joint angular velocities**, measured in rad/s:

$$\dot{q}_i = \frac{dq_i}{dt}.$$

Apply the chain rule to get the heading's time derivative:

$$\begin{aligned} \dot{\theta} &= \frac{\partial \theta}{\partial q_1}\dot{q}_1 + \frac{\partial \theta}{\partial q_2}\dot{q}_2 \\ &= \dot{q}_1 + \dot{q}_2. \end{aligned}$$

Then stack the new heading row above the position rows of (12):

$$\mathbf{J}_{\text{full}} = \begin{bmatrix} \partial\theta/\partial q_1 & \partial\theta/\partial q_2 \\ \partial x/\partial q_1 & \partial x/\partial q_2 \\ \partial y/\partial q_1 & \partial y/\partial q_2 \end{bmatrix}.$$

This is the full planar Jacobian, with angular motion first as in the 3D geometric Jacobian. Only heading about z is part of this planar task, so its angular block has one row. The spatial representation in §3.4 uses three angular rows, with its x and y rows zero for the same planar arm. In planar block form,

$$\mathbf{J}_{\text{full}} = \begin{bmatrix} \mathbf{J}_{\text{ang}} \\ \mathbf{J}_{\text{pos}} \end{bmatrix}, \qquad \mathbf{J}_{\text{ang}} = \begin{bmatrix} 1 & 1 \end{bmatrix}. \tag{14}$$

Multiplying all three rows by the same two joint angular velocities gives heading rate and tip velocity. At the worked pose, the position Jacobian is invertible, so a requested tip velocity determines both joint angular velocities. Their sum then fixes the heading rate. We cannot choose an additional heading rate independently.

For a stationary tip, the two column arrows must sum to zero after scaling by the joint angular velocities. At this pose the arrows are nonparallel, so both angular velocities must be zero. The heading rate is therefore zero too. This argument depends on the position Jacobian being invertible.

The numerical check in §5(h) uses opposite joint rates to keep heading fixed. More generally, substituting that condition into the position rows cancels their link-2 contributions:

$$\dot{q}_2 = -\dot{q}_1, \qquad \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = l_1 \begin{bmatrix} -\sin q_1 \\ \cos q_1 \end{bmatrix} \dot{q}_1.$$

Because link 1 has nonzero length, this velocity vanishes only when both joint rates vanish. The full Jacobian has at most two independent columns and cannot produce every independently requested triple of heading and position rates. The arm is **kinematically deficient for that task** (there are fewer joints than requested task freedoms). The opposite situation, more joints than task coordinates ($n > m$, the 6 × 7 Panda), is **redundancy**: many $\dot{\mathbf{q}}$ produce the same $\mathbf{V}$. Neither case has a square, invertible $\mathbf{J}$, which is the first of the reasons why week 3 poses inverse kinematics as an optimization.

For a 3D gripper, the same procedure predicts both turning and translation. At the current joint positions, each column of the full geometric Jacobian gives the gripper's angular velocity and the linear velocity of its origin when one joint moves at unit rate. Multiply each column by that joint's velocity, then add the columns' contributions. The upper three rows produce angular velocity; the lower three produce origin velocity. For the Panda, the selected origin is the flange frame; the FR3 grasp task instead uses the tool center after the fixed flange-to-tool transform. This is differential forward kinematics. It supplies the velocity map that differential IK will use in reverse.

4.9 Optional derivation: the manipulability ellipse

This derivation supports backup slide 55 and independent module exploration. In the main sequence, the column arrows already show the two available directions and their loss when the arm stretches. The ellipse and its singular values are additional ways to measure those motion gains.

Feed every joint velocity of unit norm through the two-link position Jacobian. Its singular value decomposition uses two orthogonal factors. In this subsection, $\mathbf{V}$ denotes that factor, not the spatial-velocity symbol. Let $\mathbf{w}$ be the joint velocities expressed along its columns:

$$\dot{\mathbf{q}} = \mathbf{V}\mathbf{w}, \qquad \|\dot{\mathbf{q}}\| = \|\mathbf{w}\| = 1, \qquad \mathbf{J}\dot{\mathbf{q}} = \mathbf{U}\mathbf{\Sigma}\mathbf{w}.$$

Multiplying a unit circle by the diagonal matrix $\mathbf{\Sigma}$ scales its two coordinates. The resulting ellipse has semiaxes equal to the two singular values. The orthogonal factor $\mathbf{U}$ then orients its axes in task space. So the image is an **ellipse whose semi-axes are the singular values, along the columns of $\mathbf{U}$**: long axis, the direction the tip moves fastest per unit joint speed; short axis, slowest. Its area is $\pi\sigma_1\sigma_2 = \pi|\det \mathbf{J}|$. At a singularity $\sigma_2 = 0$ and the ellipse is a line segment. Figures `fig/two_link_jacobian.png` and `fig/two_link_singular.png`, and the module in §6, draw it.

4.10 Finite differences as a check

For any φ you can evaluate, column j of the Jacobian is approximately

$$\mathbf{J}_{:,j}(\mathbf{q}) \approx \frac{\varphi(\mathbf{q} + h\mathbf{e}_j) - \varphi(\mathbf{q} - h\mathbf{e}_j)}{2h}. \tag{15}$$

Here $\mathbf{e}_j$ is the j-th unit vector and h is small (10^{-6} rad below). Use this check when implementing an analytic Jacobian. The same comparison applies to the gradients used in optimization.

4.11 Differential inverse kinematics: the square case and a preview

The Jacobian first answered a forward motion question: given joint angular velocities, what tip velocity results? To lift a mug, we instead choose an upward hand velocity and ask which joint angular velocities will produce it. For the two-link position task, a straight upward request of 0.1 m/s means

$$\dot{\mathbf{x}} = \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} \ \mathrm{m/s}.$$

At the current configuration, the Jacobian is known. The joint angular velocities are the unknowns in

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}} = \dot{\mathbf{x}}.$$

Solving this local velocity problem is **differential inverse kinematics**. It chooses motion at the current instant; it does not directly find joint angles for a distant target. Its practical value is that the unknown joint velocities enter linearly. Repeated small steps toward a target, with the Jacobian recomputed after each step, can build numerical IK without solving the full nonlinear inverse problem at once. This local method does not guarantee convergence from every starting configuration, and limits or collisions need additional treatment. In computation we solve the linear system; the inverse below expresses its unique solution when the Jacobian is square and invertible.

If $m = n$ and $\det \mathbf{J} \neq 0$, the linear system $\mathbf{J}\dot{\mathbf{q}} = \dot{\mathbf{x}}$ for a desired tip velocity $\dot{\mathbf{x}}$ has the unique solution

$$\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}}. \tag{16}$$

with $\mathbf{J}^{-1}$ from the 2×2 inverse formula of §2. At the worked bent pose, the upward request gives

$$\dot{\mathbf{q}} \approx \begin{bmatrix} 0.3415 \\ -0.5772 \end{bmatrix} \text{ rad/s}.$$

These joint angular velocities produce the requested tip velocity through the same Jacobian used for the forward calculation. Section 5(i) checks the result by hand.

As $\det \mathbf{J} \to 0$ the entries of $\mathbf{J}^{-1}$ grow like $1/\det \mathbf{J}$: for a request with a component along the weak direction (nearly along the arm) the joint rates grow without bound, and beyond the singularity that part of the solution flips sign, while a request across the arm requires a much smaller joint-speed norm. Section 5(i) compares the same request at three configurations. Optimization I develops damping to limit large proposed joint changes. First we need a selection rule for tasks with several possible joint-velocity answers.

4.11.1 Leave one motion component unspecified, then choose an answer

The square calculation specifies both components of tip velocity. At the same worked configuration, now ask only for horizontal tip velocity of 0.1 m/s. Leave vertical velocity unspecified. This changes the task; it does not change the robot or its configuration. Use only the first row of its position Jacobian:

$$\mathbf{J}_x = \begin{bmatrix} -0.4898 & -0.2898 \end{bmatrix} \text{ m/rad}.$$

The unknown joint angular velocities must satisfy one equation:

$$-0.4898\,\dot{q}_1 - 0.2898\,\dot{q}_2 = 0.1 \text{ m/s}.$$

Joint 1 can produce the request on its own, or joint 2 can produce it on its own. Two valid answers are

$$\dot{\mathbf{q}}_A \approx \begin{bmatrix} -0.2042 \\ 0 \end{bmatrix} \text{ rad/s}, \qquad \dot{\mathbf{q}}_B \approx \begin{bmatrix} 0 \\ -0.3451 \end{bmatrix} \text{ rad/s}.$$

Multiplying either answer by the one-row Jacobian gives the requested horizontal velocity. The displayed coefficients and answers are rounded. These choices generally give different vertical velocities, and both are allowed because vertical velocity was left unspecified. Requesting strictly horizontal motion would require setting both position-velocity components.

Plot joint 1's angular velocity on the horizontal axis and joint 2's angular velocity on the vertical axis. Each point now represents a joint-velocity pair, not a point occupied by the tip. All pairs satisfying this one equation lie on a line. The origin means both joints are stationary, so it fails this nonzero request.

We need a rule for selecting one answer. Distance from the origin measures the combined size of the two joint angular velocities:

$$\|\dot{\mathbf{q}}\| = \sqrt{\dot{q}_1^2 + \dot{q}_2^2}.$$

Choose the valid point nearest the origin. It satisfies the request with the smallest joint-velocity norm. Given a requested task velocity, the **pseudo-inverse** is the matrix that chooses joint velocities with the smallest task mismatch, then the smallest norm among equally good answers. For this horizontal-only example, the request is achievable.

For this nonzero row, the direction perpendicular to the solution line is the row written as a column. The closest point to the origin lies in that direction. Divide the column by the sum of the row's squared entries so that multiplying it by the row gives one:

$$(-0.4898)^2 + (-0.2898)^2 = 0.32388808.$$

$$\mathbf{J}_x^{+} = \frac{\mathbf{J}_x^{\top}}{\mathbf{J}_x\mathbf{J}_x^{\top}} \approx \frac{\begin{bmatrix} -0.4898 \\ -0.2898 \end{bmatrix}}{0.32388808} \approx \begin{bmatrix} -1.5123 \\ -0.8948 \end{bmatrix} \ \mathrm{rad/m}.$$

The denominator is a scalar because the task Jacobian has one row. Multiplication by the requested horizontal speed now gives the selected joint velocities:

$$\dot{\mathbf{q}} = \mathbf{J}_x^{+}(0.1) \approx \begin{bmatrix} -0.1512 \\ -0.0895 \end{bmatrix} \ \mathrm{rad/s}.$$

The displayed calculation uses the rounded coefficients. The code uses the unrounded Jacobian and checks this construction against `np.linalg.pinv`. This simple transpose-and-scalar-division formula applies to a nonzero one-row Jacobian.

Its selected answer has the following norm:

Writing this chosen velocity as $\dot{\mathbf{q}}_{\text{selected}}$, we obtain

$$\|\dot{\mathbf{q}}_{\text{selected}}\| \approx 0.1757 \ \mathrm{rad/s}.$$

The full position Jacobian shows what happens to the unrequested component:

$$\dot{\mathbf{p}} = \mathbf{J}_{\text{pos}}\dot{\mathbf{q}}_{\text{selected}} \approx \begin{bmatrix} 0.1000 \\ -0.0711 \end{bmatrix} \ \mathrm{m/s}.$$

The tip moves right and down. It still satisfies the horizontal-only request.

For a general Jacobian, the pseudo-inverse makes two choices in order. It first minimises the gap between the requested and achieved task velocity. Among answers tied for that smallest gap, it chooses the smallest joint-velocity norm:

$$\dot{\mathbf{q}} = \mathbf{J}^{+}\dot{\mathbf{x}}.$$

A reachable request is met exactly. An unreachable request is matched as closely as the Jacobian allows; its unachievable component is discarded. At the stretched two-link pose, this means the tangential component can survive while the radial component cannot. For a square, invertible Jacobian the pseudo-inverse agrees with the ordinary inverse. The norms are Euclidean, with task coordinates and units already chosen. A full-pose task also needs a choice of angular and linear scaling. Minimising the joint-velocity norm does not establish minimum energy or enforce joint-speed, joint-position, or collision limits.

4.11.2 Compare two answers to find motion that preserves the task

Both answers above produce the same horizontal tip velocity. Subtract them:

$$\delta\dot{\mathbf{q}} = \dot{\mathbf{q}}_B - \dot{\mathbf{q}}_A \approx \begin{bmatrix} 0.2042 \\ -0.3451 \end{bmatrix} \ \mathrm{rad/s}.$$

Their horizontal effects cancel because both original answers produced 0.1 m/s. Using the unrounded Jacobian and solutions gives

$$\mathbf{J}_x\delta\dot{\mathbf{q}} = \mathbf{J}_x\dot{\mathbf{q}}_B - \mathbf{J}_x\dot{\mathbf{q}}_A = 0.1 - 0.1 = 0.$$

Adding this difference, or a multiple of it, to any valid answer preserves its horizontal tip velocity. On the joint-velocity plot, the blue direction through the origin is parallel to the valid-answer line. It represents changes to joint velocities that contribute zero horizontal tip velocity.

All joint-velocity vectors producing zero velocity in the chosen task form that Jacobian's **null space**. In this example the Jacobian has only the horizontal row, so null-space motion may still change vertical tip velocity. Keeping the full tip position fixed would be a different task. In general, a null-space joint velocity satisfies

$$\mathbf{J}(\mathbf{q})\dot{\mathbf{q}}_{\text{null}} = \mathbf{0}.$$

This uses the same column interpretation as differential forward kinematics. Each joint contributes an effect on the task. A null-space combination makes their total effect zero.

4.11.3 Apply the same cancellation to the arm's posture

Suppose the Panda gripper is already at the desired grasp pose. Keep its position and orientation fixed while changing the arm's posture. Moving one joint alone generally moves the gripper, but several joints can move together so their effects at the gripper cancel. The elbow can move while the gripper remains in place and keeps its orientation.

The full-pose demo uses the flange frame E. Its six-component velocity is zero when the 6×7 geometric Jacobian times the joint angular velocities is zero. At a regular configuration, that Jacobian has rank 6. Seven joint velocities then leave one independent pattern of coordinated joint motion. This does not mean one particular joint is free to move arbitrarily. The pattern generally involves several joints, and it changes as the arm moves.

The number of independent null-space directions is

$$\dim \operatorname{null}(\mathbf{J}) = n - \operatorname{rank}(\mathbf{J}).$$

The position-only demo instead fixes the tool-center point G between the fingers and allows the gripper to turn. Its 3×7 position Jacobian has rank 3 at the demonstrated regular configurations, so four independent patterns of coordinated joint motion remain. The physical reference point changes from the flange to the tool center as well as the task losing its orientation requirement. Holding the flange's full pose also holds a rigidly mounted tool's full pose, but a position Jacobian must refer to the point actually being held.

These statements concern instantaneous motion at the current configuration. A finite step can drift from the target. The demonstrations in §5(k) repeatedly recompute the Jacobian and correct task error. A fixed velocity combination is not a guarantee of pose preservation over a long motion. Joint limits also bound how far the posture can change.

4.11.4 Optional calculation: remove a preferred motion's task effect

Slides 38–43 develop the physical meaning before this construction. The projection formula is retained in backup slide 54. Start with a preferred joint-velocity vector $\mathbf{z}$. It would cause the task velocity

$$\mathbf{J}\mathbf{z}.$$

This velocity is reachable because it came from actual joint rates. The pseudo-inverse finds the smallest joint rates that produce exactly that same task velocity:

$$\mathbf{J}^{+}\mathbf{J}\mathbf{z}.$$

Subtract those rates from the preferred ones. Both parts produce the same task velocity, so their difference produces zero:

$$\dot{\mathbf{q}}_{\text{null}} = \mathbf{z} - \mathbf{J}^{+}\mathbf{J}\mathbf{z} = (\mathbf{I} - \mathbf{J}^{+}\mathbf{J})\mathbf{z}, \qquad \mathbf{J}\dot{\mathbf{q}}_{\text{null}} = \mathbf{0}.$$

The matrix in parentheses projects a joint-rate vector into the null space. Adding this term to the requested-motion solution changes the joint rates without changing the achieved task velocity:

$$\dot{\mathbf{q}} = \mathbf{J}^{+}\dot{\mathbf{x}} + (\mathbf{I} - \mathbf{J}^{+}\mathbf{J})\mathbf{z}.$$

The equality follows because both subtracted terms produce the same task velocity:

$$\begin{aligned}\mathbf{J}(\mathbf{I} - \mathbf{J}^{+}\mathbf{J})\mathbf{z} &= \mathbf{J}\mathbf{z} - \mathbf{J}\mathbf{J}^{+}\mathbf{J}\mathbf{z} \\ &= \mathbf{J}\mathbf{z} - \mathbf{J}\mathbf{z} \\ &= \mathbf{0}.\end{aligned}$$

The second line uses the fact that the velocity from the preferred joint motion is achievable. This construction preserves the achieved task velocity even when the original request was unreachable. Deriving the pseudo-inverse and designing a controller that chooses the preferred motion remain later topics.

4.11.5 Turn a joint velocity into a short position command

The optional projection calculation is not needed for this step. Return to the square upward-motion request of §4.11. A position-command interface needs joint angles, whereas differential IK has returned angular velocities. Holding those velocities for a short control period gives an approximate angle update:

$$\mathbf{q}_{\text{next}} = \mathbf{q} + \Delta t\, \dot{\mathbf{q}}.$$

At the worked configuration, use the computed joint velocities and a 0.01 s period:

$$\dot{\mathbf{q}} \approx \begin{bmatrix} 0.3415 \\ -0.5772 \end{bmatrix} \text{ rad/s}.$$

$$\Delta\mathbf{q} = \Delta t\, \dot{\mathbf{q}} \approx 0.01 \begin{bmatrix} 0.3415 \\ -0.5772 \end{bmatrix} = \begin{bmatrix} 0.003415 \\ -0.005772 \end{bmatrix} \text{ rad}.$$

Rate times time has units of angle. Express the current joint positions in radians too before adding this increment. Multiplying the increment by the current position Jacobian predicts the tip displacement:

$$\Delta\mathbf{p} \approx \mathbf{J}_{\text{pos}}\Delta\mathbf{q} = \Delta t \begin{bmatrix} 0 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.001 \end{bmatrix} \text{ m}.$$

The prediction is 1 mm upward. Evaluating FK at the updated angles, using unrounded joint velocities, gives approximately $(-0.000002235, 0.000998027)$ m. The small difference appears because the Jacobian changes during the motion. After the robot moves, measure its joint positions and recompute the velocity command. This describes a position interface; a velocity interface can accept the rates directly, and torque commands need dynamics.

4.12 From a static target back to a joint change (the closing bridge)

The calculation in §4.11.5 answers a control question: a requested tip velocity, joint angular velocities, and a position command after one control period. The inverse problem of §3.3 is different. There the target is a position $\mathbf{y}^*$, nothing moves yet, and there is no time step. At a guess $\mathbf{q}$ the **remaining displacement** is

$$\mathbf{e} = \mathbf{y}^* - \varphi(\mathbf{q}). \tag{17}$$

The displacement is measured in meters. The first-order model of §2, $\varphi(\mathbf{q} + \Delta\mathbf{q}) \approx \varphi(\mathbf{q}) + \mathbf{J}(\mathbf{q})\Delta\mathbf{q}$, turns "reach $\mathbf{y}^*$" into a linear problem in the unknown joint change:

$$\mathbf{J}(\mathbf{q})\Delta\mathbf{q} \approx \mathbf{e}. \tag{18}$$

At the worked configuration, the position Jacobian is square and invertible, so we can solve this local model exactly:

$$\Delta\mathbf{q} = \mathbf{J}^{-1}\mathbf{e}.$$

For the target $(0.5, 0.2)$ m, the predicted joint change is $(-52.7^\circ, 74.1^\circ)$. After applying it, forward kinematics puts the tip about 0.17 m from the target. Section 5(l) shows the arithmetic. The linear model predicted that the whole error would be removed, but a 74° elbow change is too large for that prediction to remain accurate.

Recomputing the Jacobian and repeating the calculation reaches this target in five steps from this start. This is the Gauss–Newton iteration demonstrated in the essentials' Appendix C. Optimization I derives the general least-squares step, then adds step length and damping to control the proposed change. Its scalar cost measures the remaining position error:

$$f(\mathbf{q}) = \tfrac{1}{2}\|\mathbf{r}(\mathbf{q})\|^2, \qquad \mathbf{r}(\mathbf{q}) = \varphi(\mathbf{q}) - \mathbf{y}^* = -\mathbf{e}.$$

A zero local step does not establish that a target is unreachable. For example, the stretched arm cannot move its tip inward at first order, yet bending the elbow can reach an inward target. Section 5(l) calculates this case. A solver can therefore stop with nonzero error even when a solution exists. Optimization I develops stopping rules and explains what each outcome establishes. None of these local stopping tests alone certifies that a target is unreachable.

5. Worked example: the two-link arm at $\mathbf{q} = (30^\circ, 45^\circ)$

The link lengths are $l_1 = 0.4\,\mathrm{m}$ and $l_2 = 0.3\,\mathrm{m}$. The NumPy script prints these calculations, and an independent Drake model reproduces them using two revolute joints. Values are rounded to four decimals. Calculations use unrounded inputs, so summing the displayed entries can differ in the last digit.

(a) Evaluate every factor and intermediate product

The input is one fixed configuration:

$$l_1 = 0.4\ \text{m}, \qquad l_2 = 0.3\ \text{m}, \qquad q_1 = 30^\circ, \qquad q_2 = 45^\circ.$$

The desired output is the tip frame's position and heading in the world. The elbow angle is relative to link 1, so link 2's world heading will be the sum of the angles. The following calculation shows where that sum enters the matrix product.

(a.1) Substitute the values into all four factors

The two rotations set the link directions. The two translations follow the links in those directions. In the planar homogeneous convention of (9), the four matrices are

$$\mathbf{R}(q_1) = \begin{bmatrix} \cos 30^\circ & -\sin 30^\circ & 0 \\ \sin 30^\circ & \cos 30^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.866025 & -0.500000 & 0 \\ 0.500000 & 0.866025 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{T}(l_1, 0) = \begin{bmatrix} 1 & 0 & 0.4 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{R}(q_2) = \begin{bmatrix} \cos 45^\circ & -\sin 45^\circ & 0 \\ \sin 45^\circ & \cos 45^\circ & 0 \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.707107 & -0.707107 & 0 \\ 0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \end{bmatrix},$$

$$\mathbf{T}(l_2, 0) = \begin{bmatrix} 1 & 0 & 0.3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Displayed entries are rounded. Each following result uses the unrounded trigonometric values, so multiplying the printed decimals can differ in the last displayed digit.

(a.2) Start at the base and append the first rotation

Let $\mathbf{X}^{(s)}$ be the cumulative product after construction stage s. Start with the identity, whose origin and axes agree with the world:

$$\mathbf{X}^{(0)} = \mathbf{I}_3.$$

Appending the first rotation gives the frame at the base aligned with link 1:

$$\mathbf{X}^{(1)} = \mathbf{X}^{(0)}\mathbf{R}(q_1) \approx \begin{bmatrix} 0.866025 & -0.500000 & 0 \\ 0.500000 & 0.866025 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Its position is still the world origin. Its first column is the direction of its x-axis, which has heading 30 degrees. This stage equals the named pose of frame 1. It has not located the elbow or the tip.

(a.3) Append the first link offset

The next factor places the origin 0.4 m along the current x-axis. Multiplying its last column explicitly gives

$$\begin{bmatrix} 0.866025 & -0.500000 & 0 \\ 0.500000 & 0.866025 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.4 \\ 0 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.346410 \\ 0.200000 \\ 1 \end{bmatrix}.$$

The first two columns stay unchanged because the translation factor's rotation block is the identity. Therefore the entire cumulative matrix is

$$\mathbf{X}^{(2)} = \mathbf{X}^{(1)}\mathbf{T}(l_1, 0) \approx \begin{bmatrix} 0.866025 & -0.500000 & 0.346410 \\ 0.500000 & 0.866025 & 0.200000 \\ 0 & 0 & 1 \end{bmatrix}.$$

The origin is now the elbow, with axes still parallel to link 1's frame. This helper frame has heading 30 degrees. The next factor will align the axes with link 2.

(a.4) Append the elbow rotation

At the elbow, rotate the current axes by 45 degrees. The full multiplication is

$$\mathbf{X}^{(3)} = \mathbf{X}^{(2)}\mathbf{R}(q_2).$$

$$\mathbf{X}^{(3)} \approx \begin{bmatrix} 0.866025 & -0.500000 & 0.346410 \\ 0.500000 & 0.866025 & 0.200000 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0.707107 & -0.707107 & 0 \\ 0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

Each rotation entry is a row times a column. All four are

$$\begin{aligned} (\mathbf{X}^{(3)})_{11} &= \cos 30^\circ \cos 45^\circ - \sin 30^\circ \sin 45^\circ \\ &\approx 0.612372 - 0.353553 = 0.258819, \\ (\mathbf{X}^{(3)})_{12} &= -\cos 30^\circ \sin 45^\circ - \sin 30^\circ \cos 45^\circ \\ &\approx -0.612372 - 0.353553 \approx -0.965926, \\ (\mathbf{X}^{(3)})_{21} &= \sin 30^\circ \cos 45^\circ + \cos 30^\circ \sin 45^\circ \\ &\approx 0.353553 + 0.612372 \approx 0.965926, \\ (\mathbf{X}^{(3)})_{22} &= -\sin 30^\circ \sin 45^\circ + \cos 30^\circ \cos 45^\circ \\ &\approx -0.353553 + 0.612372 = 0.258819. \end{aligned}$$

The exact angle-sum identities give the rotation at 75 degrees. The right factor's last column is the homogeneous origin, so the position column remains the elbow position:

$$\mathbf{X}^{(2)} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.346410 \\ 0.200000 \\ 1 \end{bmatrix}.$$

The bottom row remains the homogeneous row because multiplying it by any factor in (9) returns that same row. The result is

$$\mathbf{X}^{(3)} \approx \begin{bmatrix} 0.258819 & -0.965926 & 0.346410 \\ 0.965926 & 0.258819 & 0.200000 \\ 0 & 0 & 1 \end{bmatrix}.$$

This is the named pose of frame 2. The elbow has the same position in stages 2 and 3, while the x-axis now follows link 2. No additional robot motion happened between these calculations.

(a.5) Append the second link offset and read the final pose

The tip is 0.3 m along frame 2's x-axis. Appending the last translation keeps the rotation block and changes the position column to

$$\begin{aligned} p_x &= 0.4\cos 30^\circ + 0.3\cos 75^\circ \approx 0.346410 + 0.077646 \approx 0.424056 \text{ m}, \\ p_y &= 0.4\sin 30^\circ + 0.3\sin 75^\circ \approx 0.200000 + 0.289778 \approx 0.489778 \text{ m}. \end{aligned}$$

Thus the full product is

$${}^{W}\mathbf{X}^{E} = \mathbf{X}^{(4)} = \mathbf{X}^{(3)}\mathbf{T}(l_2, 0) \approx \begin{bmatrix} 0.258819 & -0.965926 & 0.424056 \\ 0.965926 & 0.258819 & 0.489778 \\ 0 & 0 & 1 \end{bmatrix}.$$

The last column locates the tip. The first column gives its x-axis direction. The heading is 75 degrees, and the position agrees with the closed form (11). Only the final construction origin is the tip; earlier origins were the base or elbow.

Collect all four factors in one multiplication. The same result can be checked without writing the intermediate poses again:

$$\begin{aligned} {}^{W}\mathbf{X}^{E} &= \mathbf{R}(30^\circ)\mathbf{T}(0.4, 0)\mathbf{R}(45^\circ)\mathbf{T}(0.3, 0) \\ &\approx \begin{bmatrix} .8660 & -.5000 & 0 \\ .5000 & .8660 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & .4 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} .7071 & -.7071 & 0 \\ .7071 & .7071 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & .3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \\ &\approx \begin{bmatrix} .2588 & -.9659 & .4241 \\ .9659 & .2588 & .4898 \\ 0 & 0 & 1 \end{bmatrix}. \end{aligned}$$

Matrix multiplication is associative, so grouping the factors into successive stages gives this same product. The factor order remains fixed. The decimals shown are rounded; the reported final pose uses the unrounded sines and cosines. Each factor is a planar pose matrix with a two-by-two rotation block and a translation column.

Construction stage	Origin in world axes, in meters	Heading	Frame meaning
0	(0, 0)	0 degrees	World frame
1	(0, 0)	30 degrees	Frame 1, at the base and aligned with link 1
2	(0.346410, 0.200000)	30 degrees	Helper frame at the elbow, still aligned with link 1
3	(0.346410, 0.200000)	75 degrees	Frame 2, aligned with link 2
4	(0.424056, 0.489778)	75 degrees	End-effector frame E

(a.6) Apply the same completed product to coordinates

The product above was assembled from base to tip. Acting on a coordinate column, its rightmost factor acts first. Use the tip's own origin as a check:

$$ {}^{E}\widetilde{\mathbf{p}}^{E} = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}. $$

Apply the four factors from the right, keeping the original joint configuration fixed:

$$ \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{\mathbf{T}(l_2,0)} \begin{bmatrix} 0.3 \\ 0 \\ 1 \end{bmatrix} \xrightarrow{\mathbf{R}(q_2)} \begin{bmatrix} 0.3\cos 45^\circ \\ 0.3\sin 45^\circ \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.212132 \\ 0.212132 \\ 1 \end{bmatrix}, $$

$$ \begin{bmatrix} 0.212132 \\ 0.212132 \\ 1 \end{bmatrix} \xrightarrow{\mathbf{T}(l_1,0)} \begin{bmatrix} 0.612132 \\ 0.212132 \\ 1 \end{bmatrix}. $$

Finally apply the shoulder rotation. Writing the unrounded coordinates gives

$$ {}^{W}\widetilde{\mathbf{p}}^{E} = \begin{bmatrix} \cos 30^\circ(0.4 + 0.3\cos 45^\circ) - \sin 30^\circ(0.3\sin 45^\circ) \\ \sin 30^\circ(0.4 + 0.3\cos 45^\circ) + \cos 30^\circ(0.3\sin 45^\circ) \\ 1 \end{bmatrix} \approx \begin{bmatrix} 0.424056 \\ 0.489778 \\ 1 \end{bmatrix}. $$

These intermediate columns describe the same tip point relative to successive frames. The first translation gives its coordinates in frame 2. The elbow rotation gives its coordinates in the helper frame at the elbow with axes parallel to frame 1. The next translation gives its coordinates in frame 1, and the final rotation gives its world coordinates. They are not a trajectory of the tip. A direction would instead have final homogeneous coordinate zero, so neither translation factor would add its offset.

(a.7) Embed this planar pose in a 3D pose

The planar matrix acts on two coordinates and a homogeneous one. In 3D, keep z unchanged and move the translation into the fourth column:

$$ {}^{W}\mathbf{X}^{E}_{\text{3D}} \approx \begin{bmatrix} 0.258819 & -0.965926 & 0 & 0.424056 \\ 0.965926 & 0.258819 & 0 & 0.489778 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. $$

This is the same arm pose restricted to the world xy-plane. It provides the direct link to a spatial arm's 4 × 4 transform chain. It does not add a third joint or an out-of-plane degree of freedom.

(b) Closed form (11). $x = 0.34641 + 0.07765 = 0.42406 \approx 0.4241$, $y = 0.20000 + 0.28978 = 0.48978 \approx 0.4898$, heading $30^\circ + 45^\circ = 75^\circ$. Same numbers, as they must be.

(c) Inverse (6). ${}^{E}\mathbf{X}^{W}$ has position block $-\mathbf{R}^{\top}\mathbf{p} = (-0.5828, 0.2828)$: seen from the end-effector, the base is 0.58 m behind and 0.28 m to the left. The product $\mathbf{X}\mathbf{X}^{-1}$ is the identity to machine precision.

(d) Jacobian (12). With $s_1 = 0.5000, c_1 = 0.8660, s_{12} = 0.9659, c_{12} = 0.2588$:

$$ \begin{aligned} \mathbf{J}_{\text{pos}} &= \begin{bmatrix} -0.4\sin 30^\circ - 0.3\sin 75^\circ & -0.3\sin 75^\circ \\ 0.4\cos 30^\circ + 0.3\cos 75^\circ & 0.3\cos 75^\circ \end{bmatrix} \\ &\approx \begin{bmatrix} -.2000 - .2898 & -.2898 \\ .3464 + .0776 & .0776 \end{bmatrix} \\ &\approx \begin{bmatrix} -.4898 & -.2898 \\ .4241 & .0776 \end{bmatrix} \ \text{m/rad}. \end{aligned} $$

Degree labels identify the pose; the Jacobian differentiates joint angles in radians. The displayed decimals are rounded independently, and the final entries use unrounded trigonometric values.

Central finite differences (15) with $h = 10^{-6}$ give the same matrix to 3.7×10^{-11}. The first column is $(-y, x) = (-0.4898, 0.4241)$ and the second is $0.3(-0.9659, 0.2588) = (-0.2898, 0.0776)$, as §4.6 says.

For a numerical differential-FK calculation, set joint 1's angular velocity to 0.01 rad/s and joint 2's to zero. The same Jacobian gives

$$\dot{\mathbf{p}} = \mathbf{J}_{\text{pos}} \begin{bmatrix} 0.01 \\ 0 \end{bmatrix} \approx \begin{bmatrix} -0.004898 \\ 0.004241 \end{bmatrix} \text{ m/s}.$$

This means about 4.90 mm/s left and 4.24 mm/s up. In contrast, a joint-angle change with the same numerical components predicts a displacement in metres. The velocity relation is instantaneous and exact; the approximation sign here accounts for rounding.

(e) Determinant and singular values. $\det \mathbf{J} = (-0.4898)(0.0776) - (-0.2898)(0.4241) = -0.0380 + 0.1229 = 0.0849$, and (13) gives $0.4 \cdot 0.3 \cdot \sin 45^\circ = 0.12 \cdot 0.7071 = 0.0849$. The optional gain calculation gives singular values $\sigma_1 = 0.7037$, $\sigma_2 = 0.1206$ m/s per rad/s; $\sigma_1 \sigma_2 = 0.0849 = \det \mathbf{J}$. The largest and smallest singular values describe the fastest and slowest principal tip-motion directions per unit joint-speed norm. Their ratio is about 5.84. These directions need not coincide exactly with the directions along and across the arm.

(f) First order only. Turn joint 1 by 5°: the tip actually moves by $(-0.0443, 0.0351)$ m, the prediction $\mathbf{J}\Delta\mathbf{q}$ says $(-0.0427, 0.0370)$, an error of 2.5 mm (2.47 mm). Turn it by 30°: actual $(-0.3017, 0.1464)$, predicted $(-0.2564, 0.2220)$, an error of 9 cm. The Jacobian is a local statement; the 2.1 contact demo made the same point on the 7-joint arm.

(g) Singular configurations.

Rank and the lost first-order direction are part of the main explanation. The singular-value column is an optional quantitative check for the ellipse discussion.

q_2	$\det \mathbf{J}$	σ_1, σ_2	rank
0° (stretched)	0	0.7616, 0	1
10°	0.0208	0.7587, 0.0275	2
45°	0.0849	0.7037, 0.1206	2
90°	0.1200	0.5389, 0.2227	2
180° (folded)	0	0.3162, 0	1

At $q_2 = 0$ both columns point along $(-0.5, 0.866)$ and both are perpendicular to the radial direction $(0.866, 0.5)$: $\mathbf{J}^\top \hat{\mathbf{r}} = \mathbf{0}$. The main figure `fig/two_link_lost_direction.png` compares the column arrows at a bent and a stretched pose. The optional figure `fig/two_link_singular.png` also shows the ellipse and singular values.

(h) The 3 × 2 Jacobian (14) has rows $(1, 1)$, $(-0.4898, -0.2898)$, $(0.4241, 0.0776)$, angular first as in (14), and rank 2. Its three motion components depend on the same two joint rates. For a concrete check, set the rates to 1 rad/s and -1 rad/s. The heading rate is zero, but the tip moves:

$$\begin{bmatrix} \dot{\theta} \\ \dot{x} \\ \dot{y} \end{bmatrix} = \mathbf{J}_{\text{full}} \begin{bmatrix} 1 \\ -1 \end{bmatrix} \approx \begin{bmatrix} 0 \\ -.2000 \\ .3464 \end{bmatrix}.$$

This uses unrounded coefficients; the angular output is in rad/s and the linear outputs are in m/s. At this regular position configuration, fixing the tip velocity uniquely fixes the joint rates and therefore also fixes heading rate. We cannot prescribe an independent third motion component using only those two joint rates.

(i) Differential inverse kinematics (16) for a tip velocity of 0.1 m/s straight up, $\dot{\mathbf{x}} = (0, 0.1)$, at $q_1 = 30^\circ$ and three elbow angles:

The first row is the regular main-slide calculation. The comparison with the near-stretched poses is retained in backup slide 58. Its lesson remains in the main explanation: the same requested speed can require much larger joint rates when it has a component in a weak direction.

q_2	$\det \mathbf{J}$	$\dot{\mathbf{q}} = \mathbf{J}^{-1}\dot{\mathbf{x}}$ (rad/s)	$\lVert\dot{\mathbf{q}}\rVert$
45°	0.0849	$(0.3415, -0.5772)$	0.67
10°	0.0208	$(0.9254, -1.8852)$	2.10
1°	0.0021	$(7.3778, -16.9275)$	18.47

The same ten centimetres per second cost 28 times the joint speed one degree from the singularity. Check the first row by hand with the 2 × 2 inverse formula: $\mathbf{J}^{-1} \approx \frac{1}{0.0849}\begin{bmatrix} 0.0776 & 0.2898 \\ -0.4241 & -0.4898 \end{bmatrix}$, so $\dot{\mathbf{q}} = (1/0.0849)(0.2898 \cdot 0.1, -0.4898 \cdot 0.1) = (0.3414, -0.5769)$, matching to the rounding of the entries.

(j) Optional Panda singularity example. A spatial arm can lose a turning direction while retaining all three translation directions. This example uses the Panda model in `drake/01-fk-jacobian/fk_jacobian_demo.py`; the FR3 grasp example uses a separate model.

At the all-zero configuration, the flange origin is at $(0.088, 0, 0.926)$ m. This configuration lies outside the Panda model's joint-4 limit, so it is a geometric example. The Jacobian describes the flange's angular velocity and origin velocity in world axes. Its rows are angular x, y, z, followed by linear x, y, z. Its columns are joints 1 through 7:

$$\mathbf{J}(\mathbf{0}) \approx \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & -1 & 0 & -1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & -1 \\ 0 & 0.593 & 0 & -0.277 & 0 & 0.107 & 0 \\ 0.088 & 0 & 0.088 & 0 & 0.088 & 0 & 0 \\ 0 & -0.088 & 0 & 0.0055 & 0 & 0.088 & 0 \end{bmatrix}.$$

Multiplying by joint angular velocities in rad/s produces angular velocity in rad/s from the first three rows and linear velocity in m/s from the last three.

The first row is zero, so every joint-velocity combination gives zero angular velocity about world x. The matrix has rank 5. This configuration is singular for the full-pose motion task because the same Jacobian normally reaches rank 6. The three position rows still have rank 3, so the flange can move in every translation direction when turning is left unspecified.

Compare this with the ready configuration:

$$\mathbf{q} = (0, -0.785, 0, -2.356, 0, 1.571, 0.785)^\top \text{ rad}.$$

The flange is then approximately at $(0.307, 0, 0.590)$ m, and the full Jacobian has rank 6. The stretched two-link position task loses radial translation; the Panda's zero configuration loses rotation about world x. The lost direction depends on the configuration and the selected task.

The demo also provides a near-singular preset inside the joint limits. Its full Jacobian still has rank 6 at the displayed tolerance, but a weak motion direction requires large joint velocities. The optional demo diagnostics record the preset, scaling, tolerances and joint-4 sweep.

(k) Redundancy on the real arm. Several Panda postures can hold the same gripper pose. In the full-pose demonstration, the flange stays at $(0.556, 0, 0.245)$ m with its tool axis pointing down, while the elbow changes position.

Here the FK map returns the flange pose. The configurations satisfying

$$\varphi(\mathbf{q}) = \mathbf{X}^*$$

form a curve locally where the geometric Jacobian has rank 6. Seven joint velocities and six independent motion components leave one independent coordinated motion pattern:

$$\dim \operatorname{null}(\mathbf{J}) = 7 - 6 = 1.$$

That pattern generally involves several joints and changes as the arm moves. The demo recomputes the null-space direction, takes a small step and corrects pose drift. Joint limits bound how far it can continue.

The second demonstration fixes only the tool-center point between the fingers and allows the gripper to turn. Its position Jacobian has rank 3 at the demonstrated configurations, so four independent coordinated motion patterns remain:

$$\dim \operatorname{null}(\mathbf{J}_{\text{pos}}) = 7 - 3 = 4.$$

These patterns may each involve several joints. The selected reference point also changes: the full-pose task uses the flange, and the position-only task uses the tool center. Holding the flange's full pose holds a rigidly mounted tool's full pose, but holding one point does not fix the tool's orientation.

The two-link position task has two separate joint configurations for an interior target. The Panda's redundancy provides continuous local posture freedom. This matters for learning because demonstrations may use different postures for the same grasp, and averaging their joint angles need not preserve that grasp.

The scripts `self_motion_panda.py` and `tip_only_panda.py` reproduce these demonstrations. The demo reference gives their commands and controls. Its optional diagnostics record the trace and individual-joint compensation checks.

(l) One local step toward a static target. Use the position target $(0.5, 0.2)$ m, which Optimization I will also use. At the worked configuration, the tip and its remaining displacement are

$$\mathbf{p} \approx \begin{bmatrix} 0.4241 \\ 0.4898 \end{bmatrix} \text{ m}, \qquad \mathbf{e} = \mathbf{y}^* - \mathbf{p} \approx \begin{bmatrix} 0.0759 \\ -0.2898 \end{bmatrix} \text{ m}, \qquad \|\mathbf{e}\| \approx 0.2996 \text{ m}.$$

Solving the square local model gives

$$\Delta\mathbf{q} = \mathbf{J}^{-1}\mathbf{e} \approx \begin{bmatrix} -0.9201 \\ 1.2931 \end{bmatrix} \text{ rad} \approx \begin{bmatrix} -52.7^\circ \\ 74.1^\circ \end{bmatrix}.$$

Adding this change gives joint angles of about $(-22.7^\circ, 119.1^\circ)$. Evaluating forward kinematics again gives the tip position $(0.3357, 0.1437)$ m, still 0.1737 m from the target. The bridge figure shows the original arm, the target and the arm after this step. Recomputing the Jacobian and repeating reaches $(-11.5^\circ, 80.4^\circ)$ in five steps from this start, as the essentials' Appendix C demonstrates.

A second starting configuration illustrates why a zero step does not establish unreachability. At the stretched configuration $(0, 0)$, the tip is at $(0.7, 0)$ m. Choose the target $(0.5, 0)$ m, which lies inside the reachable annulus. Then

$$\mathbf{e} = \begin{bmatrix} -0.2 \\ 0 \end{bmatrix} \text{ m}, \qquad \mathbf{J} = \begin{bmatrix} 0 & 0 \\ 0.7 & 0.3 \end{bmatrix}, \qquad \mathbf{J}^\top \mathbf{e} = \mathbf{0}.$$

The local model cannot predict inward motion, so its minimum-norm least-squares step is zero. Adding a positive damping penalty also selects zero. An added null-space motion can still bend the elbow; the resulting inward displacement begins at second order. The target is reachable even though this local error-reducing step is zero.

(m) A separate FR3 example connects the current hand pose to a mug grasp

This example uses the actual FR3 description from `frankarobotics/franka_description`, release 2.9.0, commit `7aeeddc449edf8d62b594f9e36a81da53e7796f9`. The local model and provenance are in `drake/01c-fr3-grasp-chain/model/`. It describes nominal manufacturer geometry, not a calibration of the particular lab arm. The previous Panda examples keep their own model and their existing numbers.

The robot base `fr3_link0` is fixed at the world origin with matching axes. The seven joint values are exact multiples of pi:

$$\mathbf{q} = (0, -\pi/4, 0, -3\pi/4, 0, \pi/2, \pi/4) \text{ rad}.$$

The finger joints are held symmetrically open at 0.04 m each. They branch from the hand and do not add factors to the arm-to-tool-center chain. The flange frame E is `fr3_link8`. The grasp frame G is the manufacturer's `fr3_hand_tcp`. The latter is a named tool-center point, not a new arm joint.

(m.1) Read the arm's seven relative transforms from its model

The Unified Robot Description Format (URDF) file gives each joint a fixed origin transform and an axis. All seven FR3 arm joints use their local z-axis. Numbered frames in the next equation refer to `fr3_link0` through `fr3_link7`, separate from the planar example's frames. A parent-to-child factor is

$${}^{i-1}\mathbf{X}^{i}(q_i) = \begin{bmatrix} \mathbf{R}_{\text{fixed},i} & \mathbf{p}_{\text{fixed},i} \\ \mathbf{0}^\top & 1 \end{bmatrix} \begin{bmatrix} \mathbf{R}_z(q_i) & \mathbf{0} \\ \mathbf{0}^\top & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{\text{fixed},i}\mathbf{R}_z(q_i) & \mathbf{p}_{\text{fixed},i} \\ \mathbf{0}^\top & 1 \end{bmatrix}.$$

The fixed offset is expressed in the parent link's axes. The joint turns about the z-axis of the fixed joint frame. The fixed rotations therefore matter even when a joint value is zero; the axes of the spatial arm do not all point along world z.

Model joint	Fixed offset in parent axes, meters	Fixed rotation	Joint value, radians
`fr3_joint1`	$(0, 0, 0.333)$	$\mathbf{I}_3$	0
`fr3_joint2`	$(0, 0, 0)$	$\mathbf{R}_x(-\pi/2)$	$-\pi/4$
`fr3_joint3`	$(0, -0.316, 0)$	$\mathbf{R}_x(\pi/2)$	0

Model joint	Fixed offset in parent axes, meters	Fixed rotation	Joint value, radians
`fr3_joint4`	$(0.0825, 0, 0)$	$\mathbf{R}_x(\pi/2)$	$-3\pi/4$
`fr3_joint5`	$(-0.0825, 0.384, 0)$	$\mathbf{R}_x(-\pi/2)$	0
`fr3_joint6`	$(0, 0, 0)$	$\mathbf{R}_x(\pi/2)$	$\pi/2$
`fr3_joint7`	$(0.088, 0, 0)$	$\mathbf{R}_x(\pi/2)$	$\pi/4$

After joint 7, the model's fixed joint `fr3_joint8` shifts 0.107 m along link 7's z-axis to the flange. Its rotation is the identity. Thus the complete world-to-flange chain is

$$
\begin{aligned}
{}^{W}\mathbf{X}^{E}(\mathbf{q}) = {}&{}^{W}\mathbf{X}^{0}\,{}^{0}\mathbf{X}^{1}(q_1)\,{}^{1}\mathbf{X}^{2}(q_2)\,{}^{2}\mathbf{X}^{3}(q_3)\\
&{}^{3}\mathbf{X}^{4}(q_4)\,{}^{4}\mathbf{X}^{5}(q_5)\,{}^{5}\mathbf{X}^{6}(q_6)\,{}^{6}\mathbf{X}^{7}(q_7)\,{}^{7}\mathbf{X}^{E}.
\end{aligned}
$$

$$
{}^{W}\mathbf{X}^{0} = \mathbf{I}_4, \qquad {}^{7}\mathbf{X}^{E} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.107 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

This is the same block multiplication as the two-link example, repeated for spatial frames. At joint 2, the fixed parent-x rotation is minus 90 degrees and the variable local-z rotation is minus 45 degrees. Its rotation block is

$$
{}^{1}\mathbf{R}^{2} = \mathbf{R}_x(-\pi/2)\mathbf{R}_z(-\pi/4).
$$

$$
{}^{1}\mathbf{R}^{2} \approx \begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ -0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \end{bmatrix}.
$$

$$
{}^{1}\mathbf{R}^{2} \approx \begin{bmatrix} 0.707107 & 0.707107 & 0 \\ 0 & 0 & 1 \\ 0.707107 & -0.707107 & 0 \end{bmatrix}.
$$

Its relative translation is zero. Joint 1 has already placed the origin 0.333 m above the base without turning the axes. Writing both full matrices gives

$$
\begin{aligned}
{}^{W}\mathbf{X}^{2} &= {}^{W}\mathbf{X}^{1}\,{}^{1}\mathbf{X}^{2} \\
&= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.333 \\ 0 & 0 & 0 & 1 \end{bmatrix}
\begin{bmatrix} c & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ c & -c & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\
&= \begin{bmatrix} c & c & 0 & 0 \\ 0 & 0 & 1 & 0 \\ c & -c & 0 & 0.333 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad c = \frac{\sqrt{2}}{2}.
\end{aligned}
$$

The first matrix's top-left identity block copies the child rotation. Its third row adds the base height to the child's zero translation. For example, entry $(3,4)$ is

$$
0 \cdot 0 + 0 \cdot 0 + 1 \cdot 0 + 0.333 \cdot 1 = 0.333.
$$

The numerical cumulative pose is

$$
{}^{W}\mathbf{X}^{2} \approx \begin{bmatrix} 0.707107 & 0.707107 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0.707107 & -0.707107 & 0 & 0.333 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

At joint 3 the variable angle is zero, but its fixed geometry is not the identity. The table gives a minus 0.316 m parent-y offset and a 90-degree rotation about parent x:

$$
{}^{2}\mathbf{X}^{3}(0) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & -0.316 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

The new position is the previous position plus the rotated local offset:

$$
\begin{aligned}
{}^{W}\mathbf{p}^{3} &= {}^{W}\mathbf{p}^{2} + {}^{W}\mathbf{R}^{2} \begin{bmatrix} 0 \\ -0.316 \\ 0 \end{bmatrix} \\
&\approx \begin{bmatrix} 0 \\ 0 \\ 0.333 \end{bmatrix} + \begin{bmatrix} -0.223446 \\ 0 \\ 0.223446 \end{bmatrix} \approx \begin{bmatrix} -0.223446 \\ 0 \\ 0.556446 \end{bmatrix} \ \mathrm{m}.
\end{aligned}
$$

The complete product also rotates the axes:

$$
{}^{W}\mathbf{X}^{3} = {}^{W}\mathbf{X}^{2}\,{}^{2}\mathbf{X}^{3}(0) \approx \begin{bmatrix} 0.707107 & 0 & -0.707107 & -0.223446 \\ 0 & 1 & 0 & 0 \\ 0.707107 & 0 & 0.707107 & 0.556446 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

A zero joint angle therefore does not remove the link's fixed offset or fixed rotation. The supplied code prints every relative factor and every world-frame intermediate product. Continuing through the fixed flange offset at the stated configuration gives

$$
{}^{W}\mathbf{X}^{E}(\mathbf{q}) \approx \begin{bmatrix} 0.707107 & -0.707107 & 0 & 0.306891 \\ -0.707107 & -0.707107 & 0 & 0 \\ 0 & 0 & -1 & 0.590282 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

(m.2) Append the fixed hand transform

The model mounts the hand with a rotation of minus 45 degrees about the flange z-axis. Its tool-center point lies 0.1034 m along that axis. These two fixed factors give

$$
{}^{E}\mathbf{X}^{G} = \begin{bmatrix} \mathbf{R}_z(-\pi/4) & \mathbf{0} \\ \mathbf{0}^{\top} & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0.1034 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

$$
{}^{E}\mathbf{X}^{G} \approx \begin{bmatrix} 0.707107 & 0.707107 & 0 & 0 \\ -0.707107 & 0.707107 & 0 & 0 \\ 0 & 0 & 1 & 0.1034 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

The offset is in flange axes. At this configuration the flange z-axis points down, so the tool-center point is below the flange:

$$
\begin{aligned}
{}^{W}\mathbf{p}^{G} &= {}^{W}\mathbf{p}^{E} + {}^{W}\mathbf{R}^{E} \begin{bmatrix} 0 \\ 0 \\ 0.1034 \end{bmatrix} \\
&\approx \begin{bmatrix} 0.306891 \\ 0 \\ 0.590282 \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -0.1034 \end{bmatrix} = \begin{bmatrix} 0.306891 \\ 0 \\ 0.486882 \end{bmatrix} \ \mathrm{m}.
\end{aligned}
$$

The rotation blocks also multiply. The full current gripper pose is

$$
{}^{W}\mathbf{X}^{G}(\mathbf{q}) = {}^{W}\mathbf{X}^{E}(\mathbf{q})\,{}^{E}\mathbf{X}^{G} \approx \begin{bmatrix} 1 & 0 & 0 & 0.306891 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0.486882 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

The tool frame's x-axis points along world x, its y-axis along negative world y, and its z-axis down. This is the current pose computed from the joints. The mug has not entered that calculation.

(m.3) Compose the supplied mug pose with a chosen grasp

The scene supplies an upright mug with its body center 0.10 m above the world origin and its handle along its local x-axis. Its world yaw is 30 degrees. Its idealized body has radius 0.035 m and height 0.08 m; its bottom rests at the table top, 0.06 m above the origin. The scene file gives

$$
\begin{aligned}
{}^{W}\mathbf{X}^{M} &= \begin{bmatrix} \mathbf{R}_z(\pi/6) & (0.50, 0.10, 0.10)^{\top} \\ \mathbf{0}^{\top} & 1 \end{bmatrix} \\
&\approx \begin{bmatrix} 0.866025 & -0.500000 & 0 & 0.50 \\ 0.500000 & 0.866025 & 0 & 0.10 \\ 0 & 0 & 1 & 0.10 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
\end{aligned}
$$

Choose a gripper target centered on the mug, with its z-axis pointing down and its x-axis following the mug's handle direction. That choice is the object-relative grasp transform

$$
{}^{M}\mathbf{X}^{G^*} = \begin{bmatrix} \mathbf{R}_x(\pi) & \mathbf{0} \\ \mathbf{0}^{\top} & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

The demo initially shows G^* alone; its controls can isolate either frame. **Compare M and G*** overlays them at their true shared origin, with longer dashed mug axes and shorter solid grasp axes. The positive x-axis is shared, while y and z reverse. The teal and orange origin markers identify the mug and grasp respectively when each is shown alone.

Its zero translation places the target tool-center point at the mug frame's origin. It does not say that every grasp of a mug belongs there. It is the specified candidate for this lesson. Multiplication gives its world pose:

$$
{}^{W}\mathbf{X}^{G^*} = {}^{W}\mathbf{X}^{M}\,{}^{M}\mathbf{X}^{G^*} \approx \begin{bmatrix} 0.866025 & 0.500000 & 0 & 0.50 \\ 0.500000 & -0.866025 & 0 & 0.10 \\ 0 & 0 & -1 & 0.10 \\ 0 & 0 & 0 & 1 \end{bmatrix}.
$$

The current and desired poses now refer to the same kind of gripper frame and both are expressed in world axes. Their world-frame position difference is

$$
{}^{W}\mathbf{p}^{G^*} - {}^{W}\mathbf{p}^{G} \approx \begin{bmatrix} 0.193109 \\ 0.100000 \\ -0.386882 \end{bmatrix} \text{ m}.
$$

The target orientation also differs. Subtracting these positions does not choose joint angles, and placing a target marker at the mug does not establish a successful grasp.

(m.4) Construct an approach target in the desired gripper's axes

1. Choose the offset. The scene chooses a stand-off distance of 0.10 m along negative target-gripper z. The grasp z-axis points down, so this offset places the approach above it.

$$d = 0.10\ \text{m}, \qquad {}^{G^*}\mathbf{X}^{G^*_{\text{app}}} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -0.10 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

2. Compose the poses. Append the offset to the world grasp pose:

$${}^{W}\mathbf{X}^{G^*_{\text{app}}} = {}^{W}\mathbf{X}^{G^*}\,{}^{G^*}\mathbf{X}^{G^*_{\text{app}}}.$$

$$\underbrace{\begin{bmatrix} 0.866025 & 0.500000 & 0 & 0.50 \\ 0.500000 & -0.866025 & 0 & 0.10 \\ 0 & 0 & -1 & 0.10 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{{}^{W}\mathbf{X}^{G^*}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & -0.10 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{{}^{G^*}\mathbf{X}^{G^*_{\text{app}}}}$$

$$\approx \underbrace{\begin{bmatrix} 0.866025 & 0.500000 & 0 & 0.50 \\ 0.500000 & -0.866025 & 0 & 0.10 \\ 0 & 0 & -1 & 0.20 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{{}^{W}\mathbf{X}^{G^*_{\text{app}}}}.$$

For the height, multiply row 3 of the left factor by column 4 of the right factor:

$$0 \cdot 0 + 0 \cdot 0 + (-1)(-0.10) + 0.10 \cdot 1 = 0.20.$$

The orientation is unchanged because the relative rotation is the identity. The complete scene chain is still

$${}^{W}\mathbf{X}^{G^*_{\text{app}}} = {}^{W}\mathbf{X}^{M}\,{}^{M}\mathbf{X}^{G^*}\,{}^{G^*}\mathbf{X}^{G^*_{\text{app}}}.$$

3. Ask for the joints. The pose product defines the approach request. Inverse kinematics seeks joint values satisfying

$${}^{W}\mathbf{X}^{G}(\mathbf{q}) = {}^{W}\mathbf{X}^{G^*_{\text{app}}}.$$

The current pose comes from the joints; the target comes from the supplied scene and chosen offset. Computing the target has not found the joints or a path to it.

The supplied code checks the chain against Drake and reproduces the target products. The FR3 example guide records the model and verification details. Section 8 explains how these frame definitions support later perception and motion problems.

6. Reproduce and explore the examples

The supplied scripts reproduce the worked calculations. Run them from the repository root with the course environment described in SETUP.md.

Supplied example	What to check
Two-link calculations	Compare the transform chain with the closed form, check the Jacobian by finite differences, and reproduce the differential-IK and static-target steps.
Transform-chain walkthrough	Follow every relative factor and cumulative pose in §5(a).
FR3 grasp chain	Reproduce the current tool pose, angular columns, desired grasp and approach target in §5(m) and §3.4.
Panda FK and Jacobian	Compare the position and spatial Jacobians at the ready and zero configurations.
Full-pose self-motion and position-only motion	Compare how much posture freedom remains under the two task definitions.

For example, these commands print the two-link and FR3 calculations and then exit:

```
.venv/bin/python \
  lectures/02-D2-transforms-fk-jacobian/code/two_link.py

.venv/bin/python drake/01c-fr3-grasp-chain/fr3_grasp_chain.py
```

Add `--meshcat` to a Drake demonstration command to open its interactive scene. Open the URL printed by that process. The Drake guide lists the commands and controls; the FR3 guide also explains link colors, opacity and the mug/grasp frame comparison.

Use the following observations to connect the demonstrations to the calculations:

- In the two-link browser module, predict what happens to the two Jacobian arrows when the elbow straightens. Select **Stretched**. The arrows become parallel, so the position task loses one instantaneous motion direction.
- In the FR3 scene, change one joint and compare the current tool frame with the desired grasp frame. Only the current tool pose changes; the target still comes from the mug and the chosen grasp. **Compare M and G*** uses dashed mug axes and solid grasp axes at their shared origin. Their x-axes agree, while y and z point in opposite directions.

- Compare the two Panda self-motion scenes. Keeping the full pose fixed preserves both position and orientation. Keeping only the tool-center point fixed allows turning.

The two-link module notes describe its optional velocity and transform-order experiments. The corresponding static examples in §§4–5 supply the calculations when a live scene is unavailable.

The supplied RAI scene describes the same planar arm, but has not been executed in RAI on this machine. Use the verified NumPy and Drake examples to reproduce this reading's results.

7. Common failure modes

- **Multiplying in the wrong order.** ${}^{W}\mathbf{X}^{E} = {}^{W}\mathbf{X}^{J_1}\mathbf{X}(q_1)\cdots$ reads from the world inward; a new transform on the **right** is expressed in the body's current frame, on the **left** in the world frame. Test with one joint: $\mathbf{R}(q_1)\mathbf{T}(l_1, 0)$ puts the elbow at $(l_1 \cos q_1, l_1 \sin q_1)$; $\mathbf{T}(l_1, 0)\mathbf{R}(q_1)$ puts it at $(l_1, 0)$ and only turns its axes.
- **Forgetting the 1.** A 4×4 transform applied to a 3-vector is a shape error in NumPy; applied to $(\mathbf{p}, 0)$ it rotates but does not translate, which is correct for a **direction** and wrong for a **point**. Keep the two apart deliberately.
- **Using an invalid rotation block.** The transpose shortcut in (6) requires a valid rotation matrix. A generic matrix inverse can invert an invalid pose matrix if that matrix is invertible, but it cannot make the geometry a valid rigid transform. Check orthogonality and determinant 1. For a valid pose, (6) exposes the structure and avoids a generic matrix inversion.
- **Rows versus columns of $\mathbf{J}$.** $\mathbf{J}$ is $m \times n$: rows are task coordinates, columns are joints. $\mathbf{J}\dot{\mathbf{q}}$ needs an n-vector; a 6×7 Jacobian times a 6-vector is a shape error, and in the 2×2 case there is no error message at all.
- **"Singular means stuck."** The arm is not stuck; it can still move. It cannot move the tip in one particular direction at first order, and near the singularity the joint rates needed for that direction explode (§5 (i)). Near-singular amplification concerns requested motion in a weak direction. At exact singularity the pseudo-inverse instead drops the unreachable component; sideways motion can remain achievable.
- **Trusting $\det \mathbf{J}$ alone.** A small determinant can mean short columns rather than parallel ones; $\sigma_{\min}$ and the rank tell the truth (§2). For non-square $\mathbf{J}$ there is no determinant.
- **Degrees in code.** `np.sin(30)` is $\sin(30\,\text{rad})$. Convert once, at the boundary.
- **Finite-difference step.** $h = 10^{-6}$ rad is safe here; $h = 10^{-12}$ loses digits to rounding, $h = 0.1$ is no longer a derivative (§5 (f)).

8. Where this sits in the stack

Forward kinematics predicts the current tool pose. The Jacobian predicts its local motion. Optimization I uses these maps to choose joint changes toward a target; Optimization II adds joint limits and clearance constraints.

The same maps connect learned commands to the robot. The required conversion depends on what the policy outputs:

Policy output	Robot-side conversion	Required conventions or checks
Joint positions or deltas	Track the positions; add deltas to the current joint positions.	Joint order, units, bounds and safety checks.
End-effector pose or pose change	Form the desired pose, then solve IK for the selected tool frame.	Expression frame, rotation-update convention, reachability, posture and collision checks.
End-effector velocity	Solve differential IK; integrate if the interface accepts positions.	Selected origin, Jacobian expression frame, units, joint-rate and position limits.

A joint-position command needs no IK conversion of that action, although other controller components may still use geometry. The action format alone does not establish what the network has learned internally or whether it transfers to another robot.

For a pose change, follow the interface's rotation and translation convention. A body-frame rigid transform increment composes on the right of the current pose; a world-frame rigid transform increment composes on the left. Some interfaces instead add a position change and update rotation separately.

8.1 Extend the same grasp example

The FR3 example separates the robot's current tool pose from the scene-defined target. Later methods can change how we estimate or reach that target while keeping the same frame definitions. An IK solver seeks the joints. A trajectory optimizer chooses a sequence of configurations under specified costs and constraints. Motion between sampled configurations needs its own collision checks. A perception system supplies the mug-pose estimate.

For example, let C be a camera frame. If the camera estimates the mug pose relative to itself, the camera-to-world transform places that estimate in the existing grasp chain:

$$ {}^{W}\mathbf{X}^{G^*} = {}^{W}\mathbf{X}^{C}\,{}^{C}\mathbf{X}^{M}\,{}^{M}\mathbf{X}^{G^*}. $$

The current demo supplies the mug pose exactly. It has not implemented this estimator, solved the grasp's IK problem, or checked a path or contact. The fixed tool mounting and chosen object-relative grasp remain explicit inputs to those later problems.

9. Further reading (optional, never load-bearing)

- Russ Tedrake, *Robotic Manipulation*, chapter 3 (kinematics, differential kinematics, the Jacobian-pseudo-inverse controller that week 3 derives properly).
- Kevin Lynch and Frank Park, *Modern Robotics*, chapters 3–5 (rigid-body motions, forward kinematics with the product of exponentials, velocity kinematics and singularities).
- Marc Toussaint, *Robotics* lecture notes (TU Berlin), chapter "Kinematics": the $\frac{\partial \mathbf{p}}{\partial q_i} = \hat{\mathbf{z}}_i \times (\mathbf{p} - \mathbf{p}_i)$ column formula and RAI's conventions.
- The Spring 2026 robotics-fundamentals deck supplies the original transform and composition treatment (pp.19–20), kinematic-tree examples (pp.26–28), and velocity/Jacobian material (pp.30–36). Its notation is translated into this course's frame convention here.

10. What remains beyond this lesson

The local models explain how to propose motion. They do not guarantee convergence to a static target, enforce limits, or check collisions. Optimization I develops the step and its stopping rules; Optimization II introduces constraints.

Those optimization examples initially use position error. Full-pose optimization also needs an orientation error and its derivative. The geometric angular block describes instantaneous turning, but it is not generally the derivative of an arbitrary finite orientation error. The long kinematics reading develops rotation-matrix derivatives; a full-pose solver must additionally specify its error convention and expression frame.